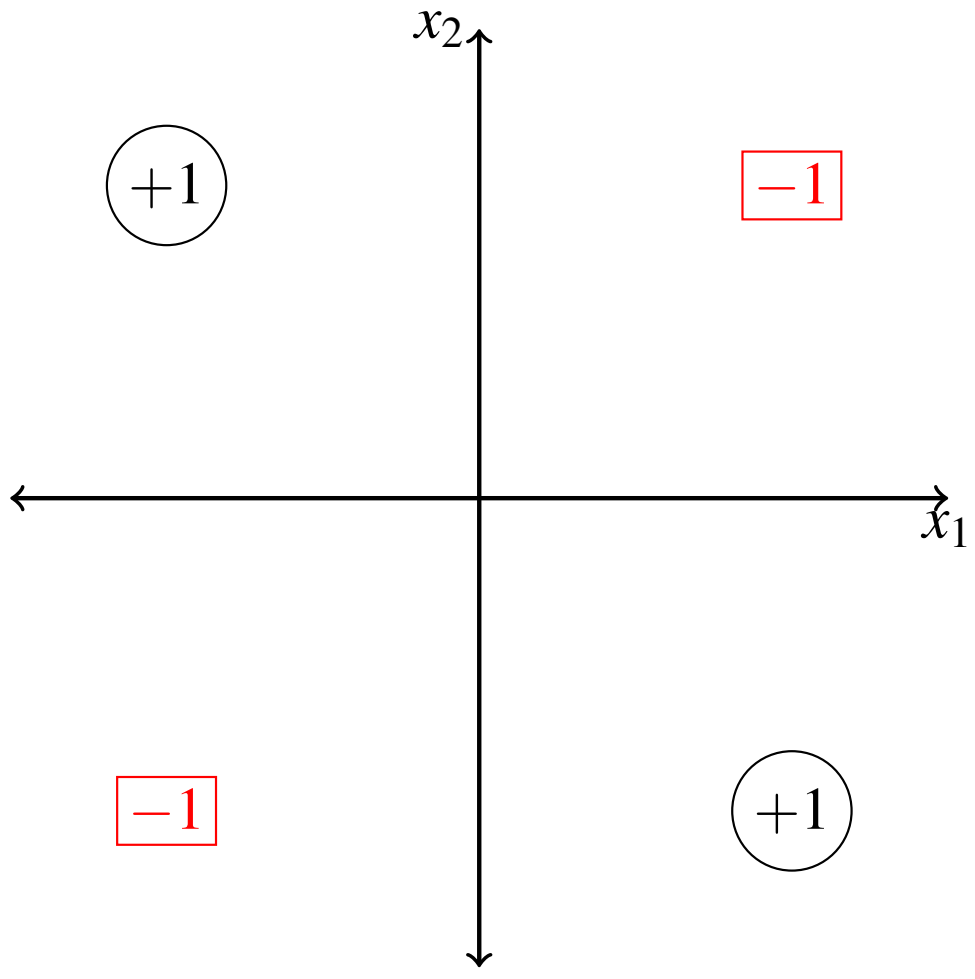
# Problema XOR con 3 neuronas

Diego Milone

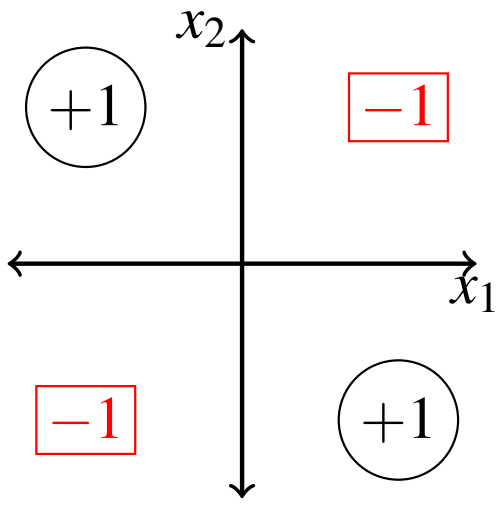
Inteligencia Computacional
Departamento de Informática

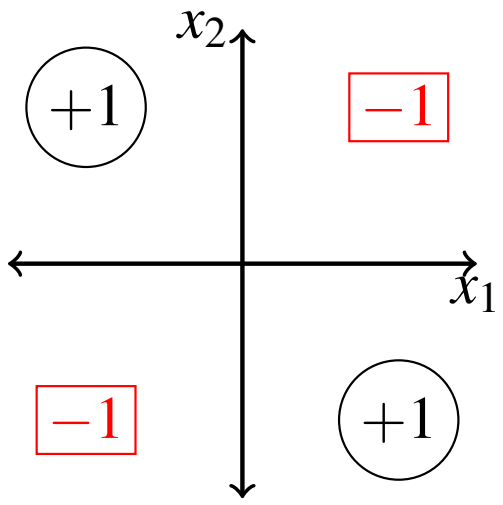
FICH-UNL

# ¿Cómo podríamos resolver el problema XOR?

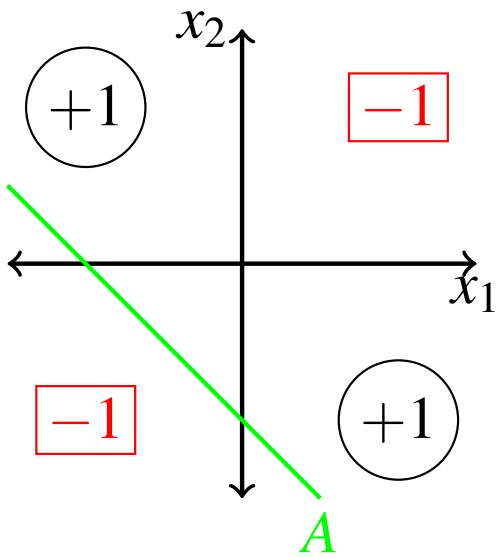


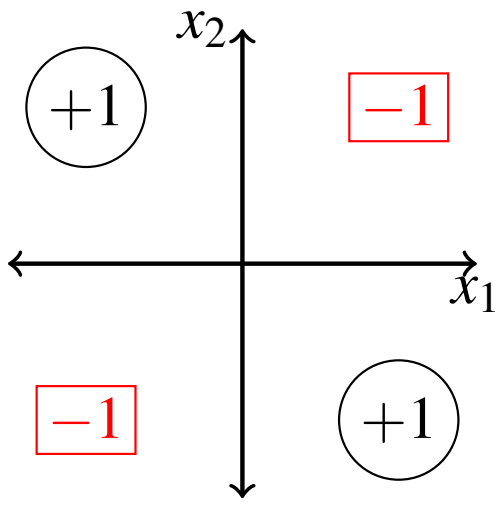
# Primera capa: perceptrones A y B



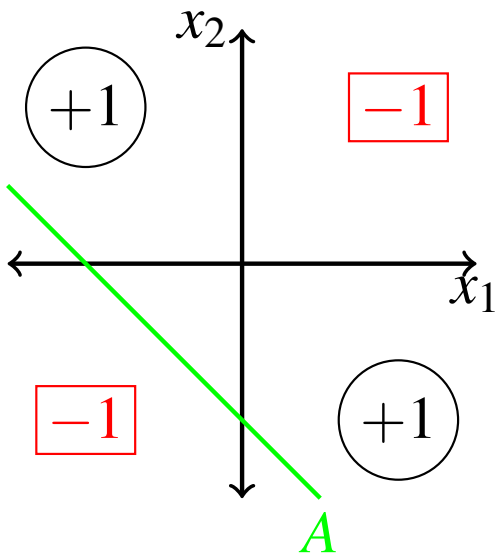



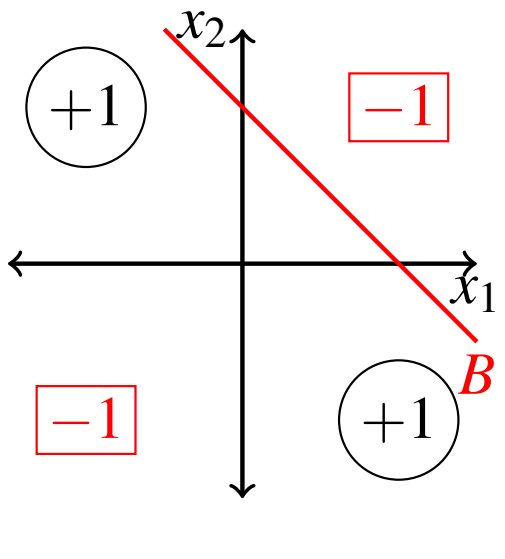
# Primera capa: perceptrones A y B



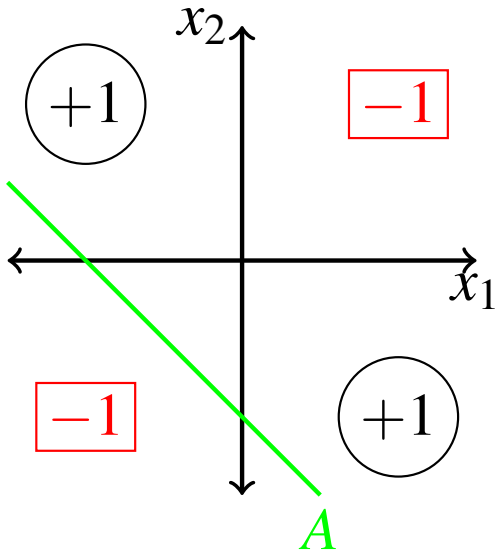



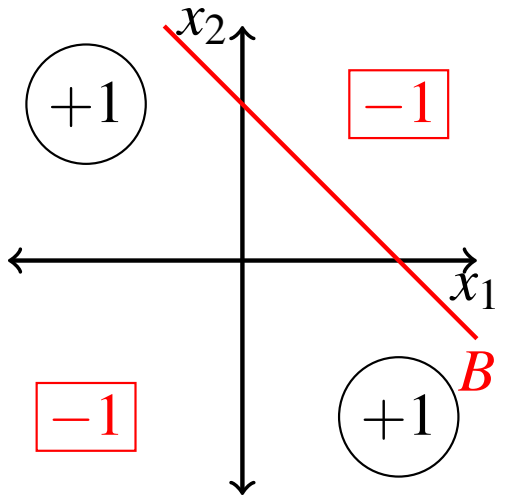
# Primera capa: perceptrones A y B

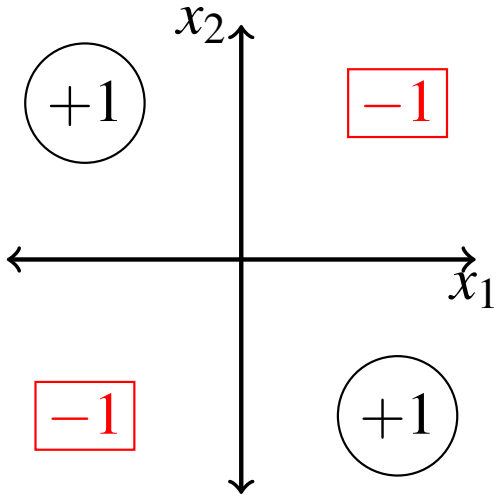


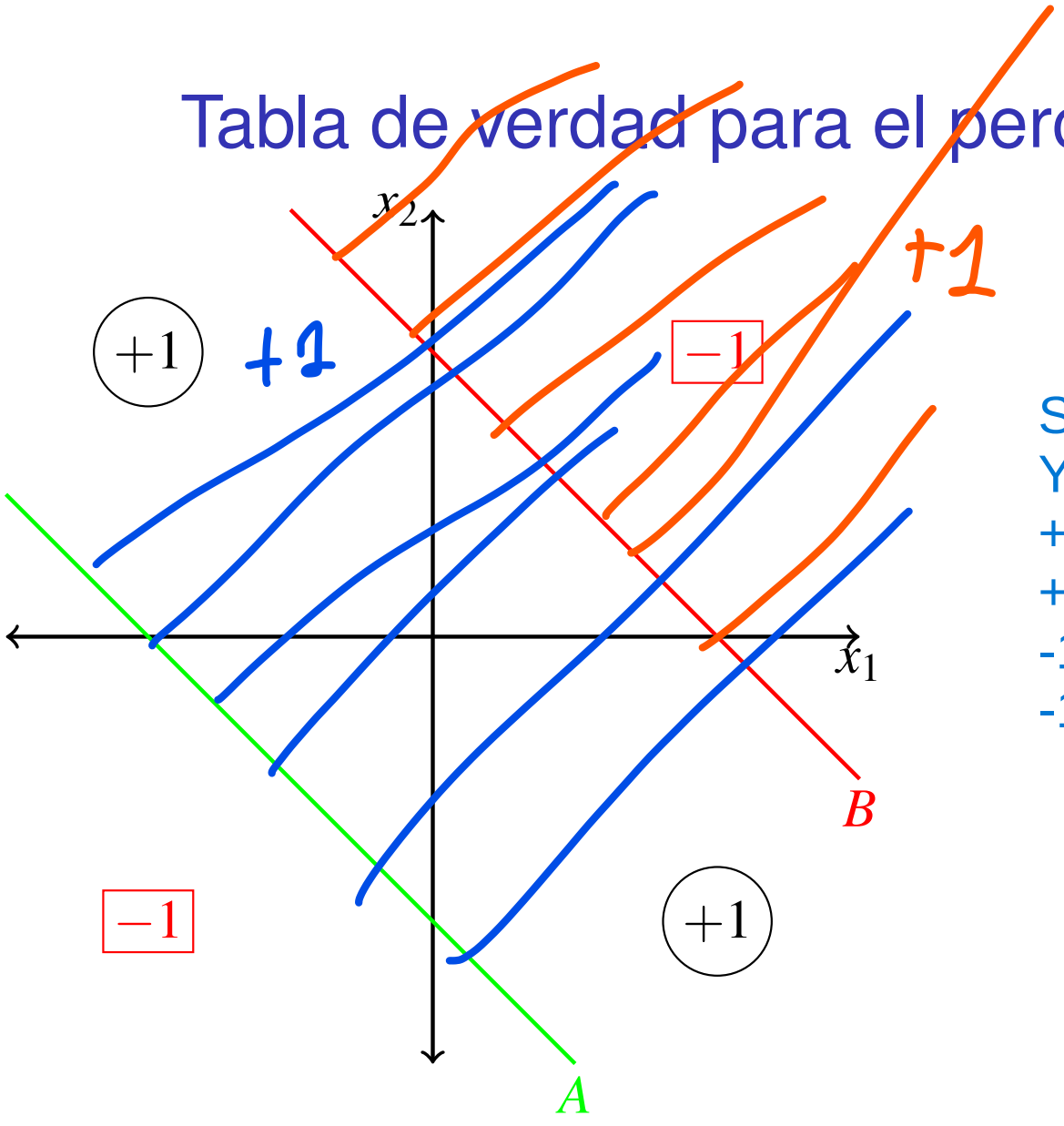
# Segunda capa: perceptron C



# Tabla de verdad para el perceptron C




graficamos para
Ya =+1
Yb = 1

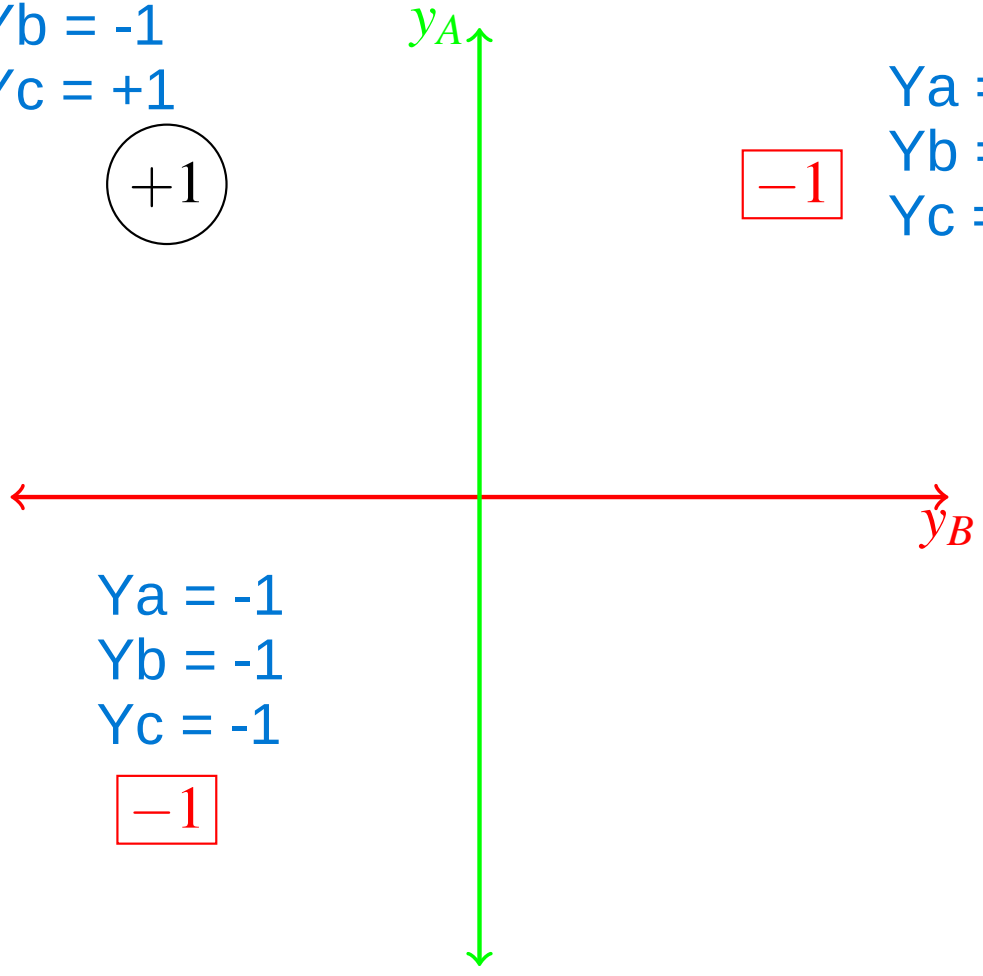
Salidas

| Ya | Yb | Yc |
|---|---|---|
| +1 | +1 | -1 |
| +1 | -1 | 1 |
| -1 | +1 | x |
| -1 | -1 | -1 |

# Tabla de verdad para el perceptron C

Ya = +1
Yb = -1
Yc = +1

$+1$



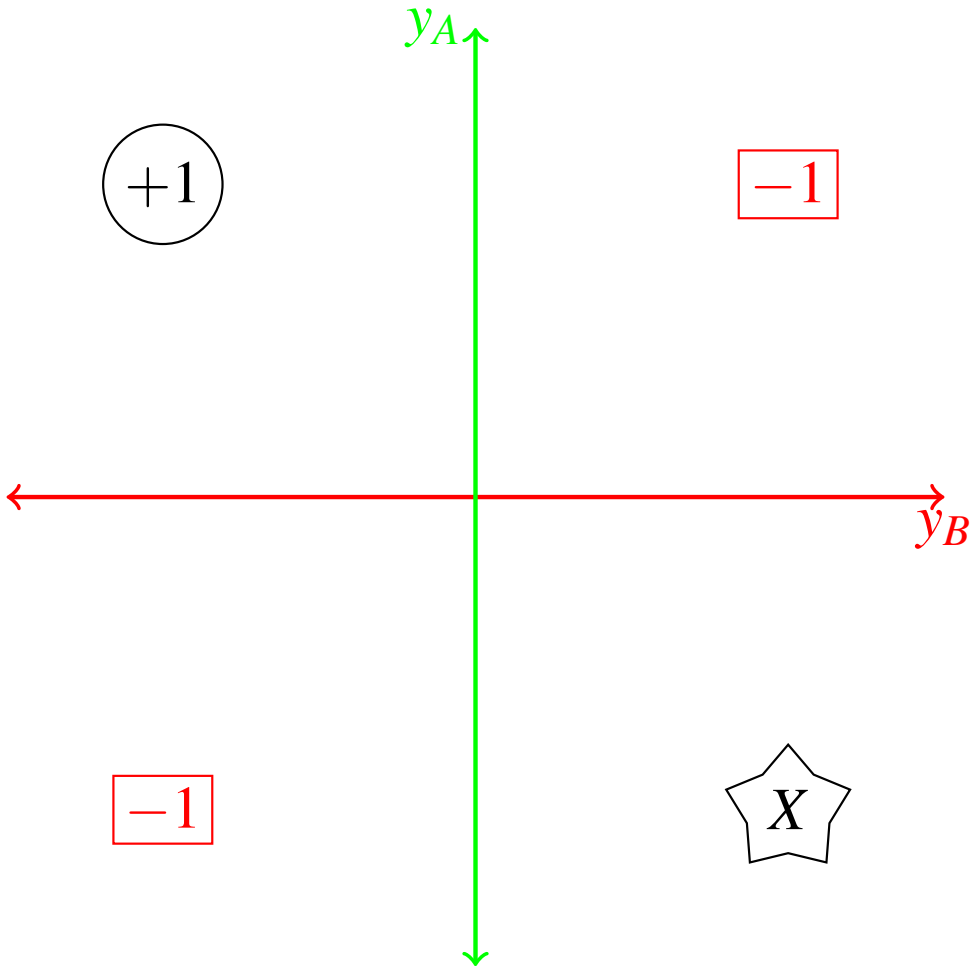

Ya = +1
Yb = +1
Yc = -1

$-1$

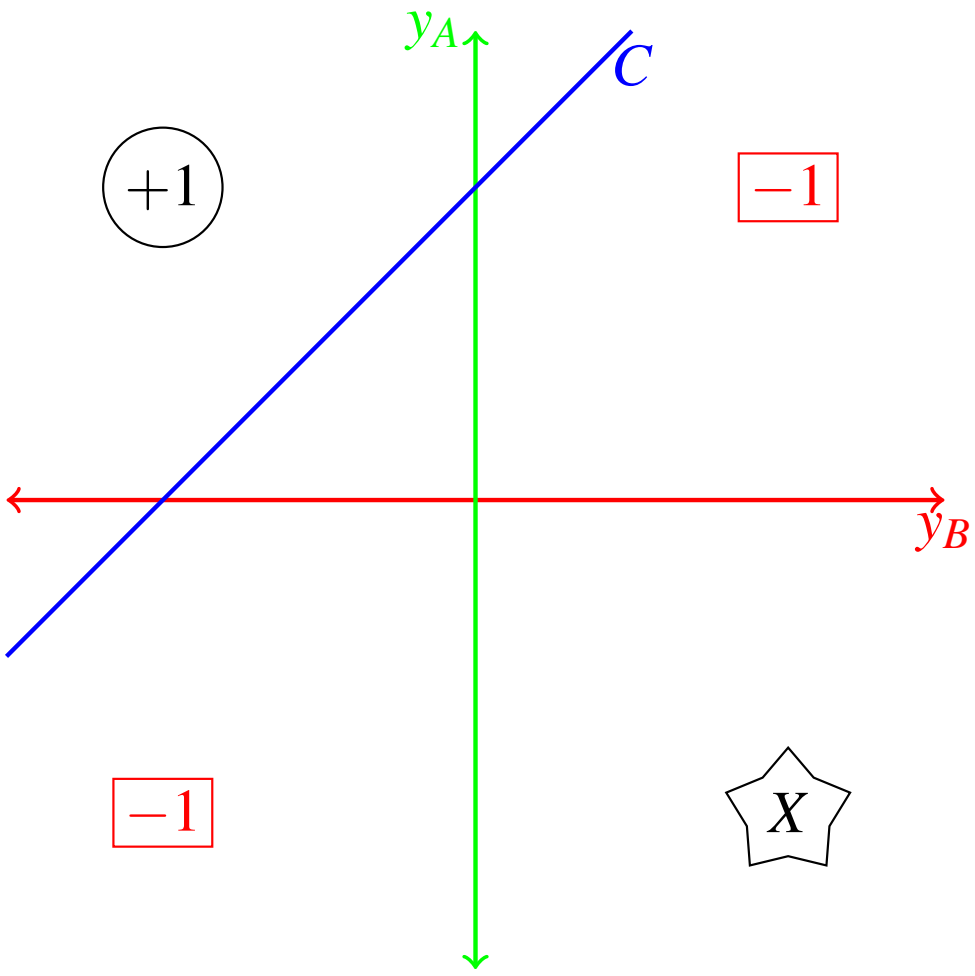
Ya = -1
Yb = -1
Yc = -1

$-1$

# Tabla de verdad para el perceptron C



# Tabla de verdad para el perceptron C



# Nuestra primera red neuronal

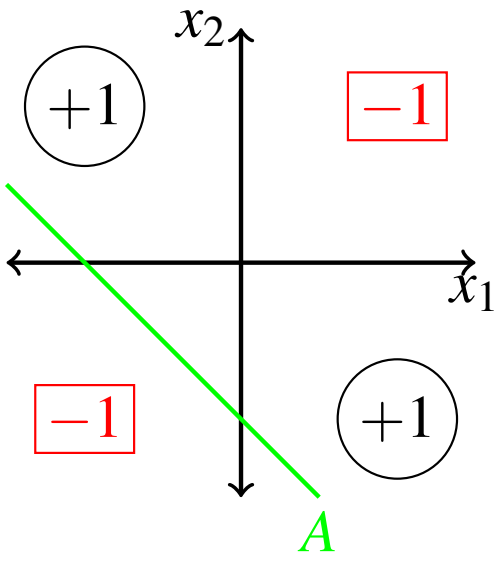
## Un perceptrón multicapa con 3 neuronas

Diego Milone

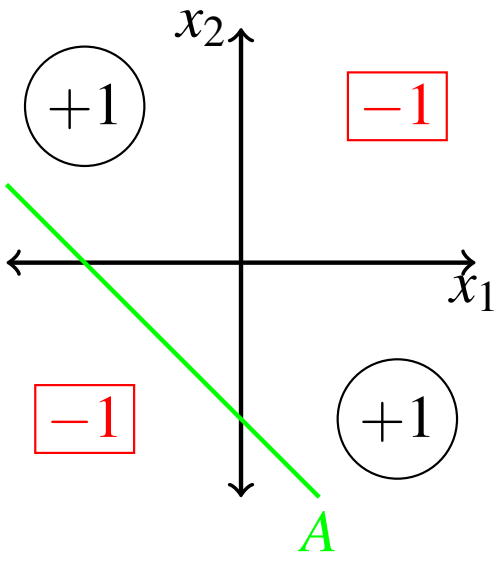
Inteligencia Computacional
Departamento de Informática

FICH-UNL

# Combinación de perceptrones simples: A



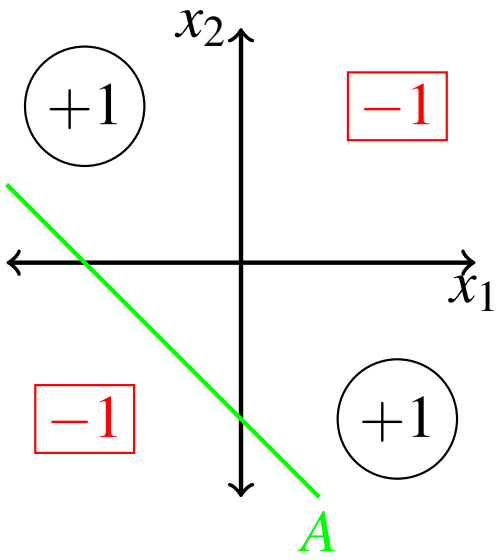
# Combinación de perceptrones simples: A




Perceptrón A: $x_2 = -1 - x_1$

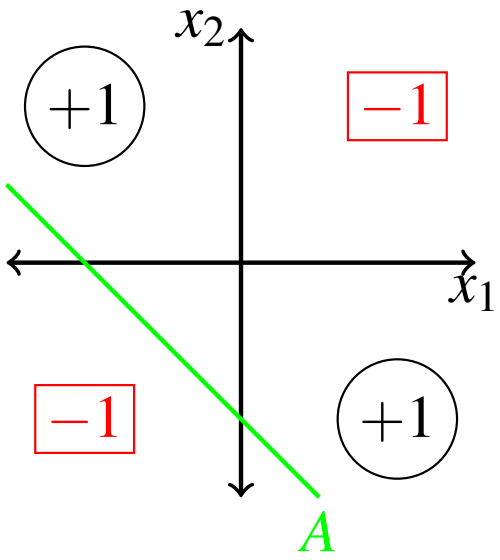
la recta: pendiente -1 , cuando x1 se hace 0 vale -1

# Combinación de perceptrones simples: A




Perceptrón A: $x_2 = -1 - x_1 = \frac{w_{A0}}{w_{A2}} - \frac{w_{A1}}{w_{A2}} x_1$

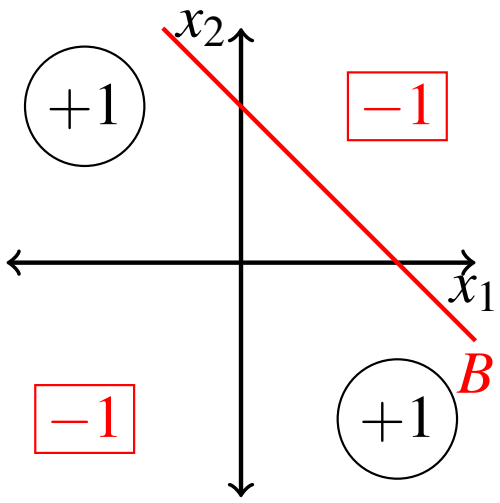
# Combinación de perceptrones simples: A



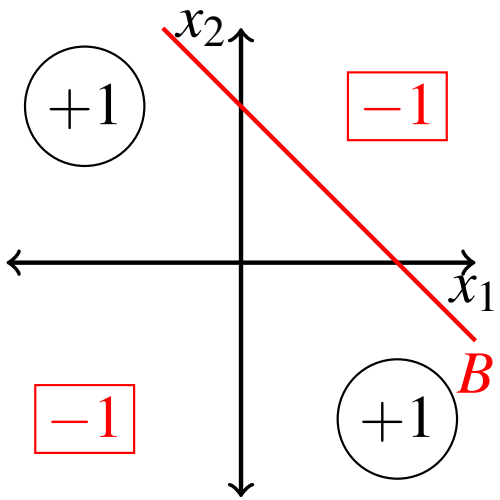

Perceptrón A: $x_2 = -1 - x_1 = \frac{w_{A0}}{w_{A2}} - \frac{w_{A1}}{w_{A2}} x_1$

$$\rightarrow \left\{ \begin{array}{l} w_{A0} = -1 \\ w_{A1} = +1 \\ w_{A2} = +1 \end{array} \right\} \rightarrow y_A = \mathrm{sgn}(x_2 + x_1 + 1)$$

# Combinación de perceptrones simples: B



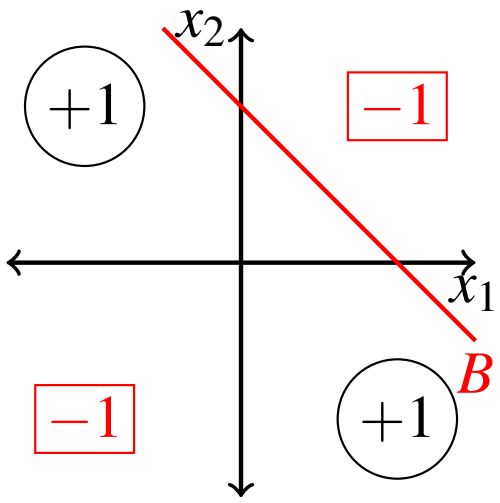
# Combinación de perceptrones simples: B




Perceptrón B: $x_2 = +1 - x_1$

cuando x1 se hace 0, x2 pasa por +1

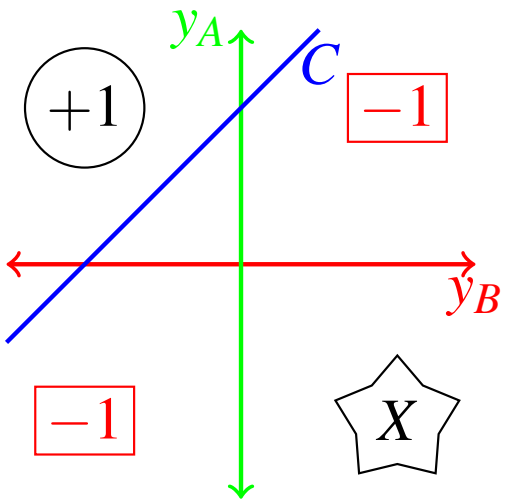
# Combinación de perceptrones simples: B



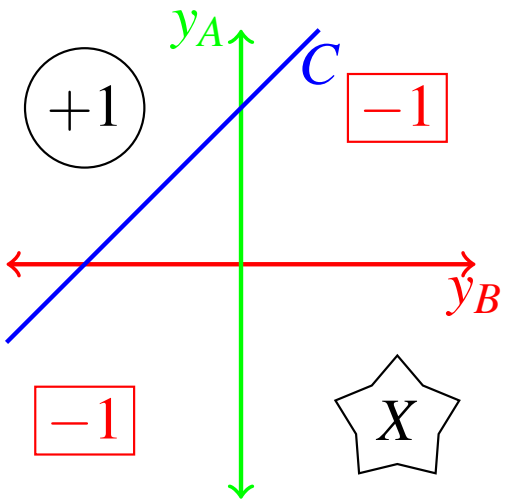

Perceptrón B: $x_2 = +1 - x_1$

$$\rightarrow \left\{ \begin{array}{l} w_{B0} = +1 \\ w_{B1} = +1 \\ w_{B2} = +1 \end{array} \right\} \rightarrow y_B = \text{sgn}(x_2 + x_1 - 1)$$

# Combinación de perceptrones simples: C



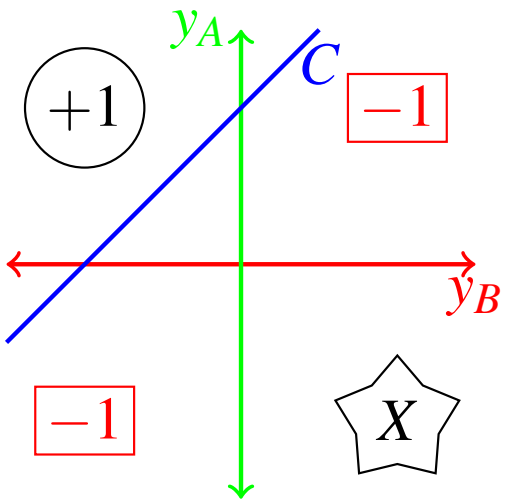
# Combinación de perceptrones simples: C




Perceptrón C: $y_A = +1 + y_B$

recibe como entradas las salidas de los dos perceptrones anteriores

# Combinación de perceptrones simples: C




Perceptrón C: $y_A = +1 + y_B$

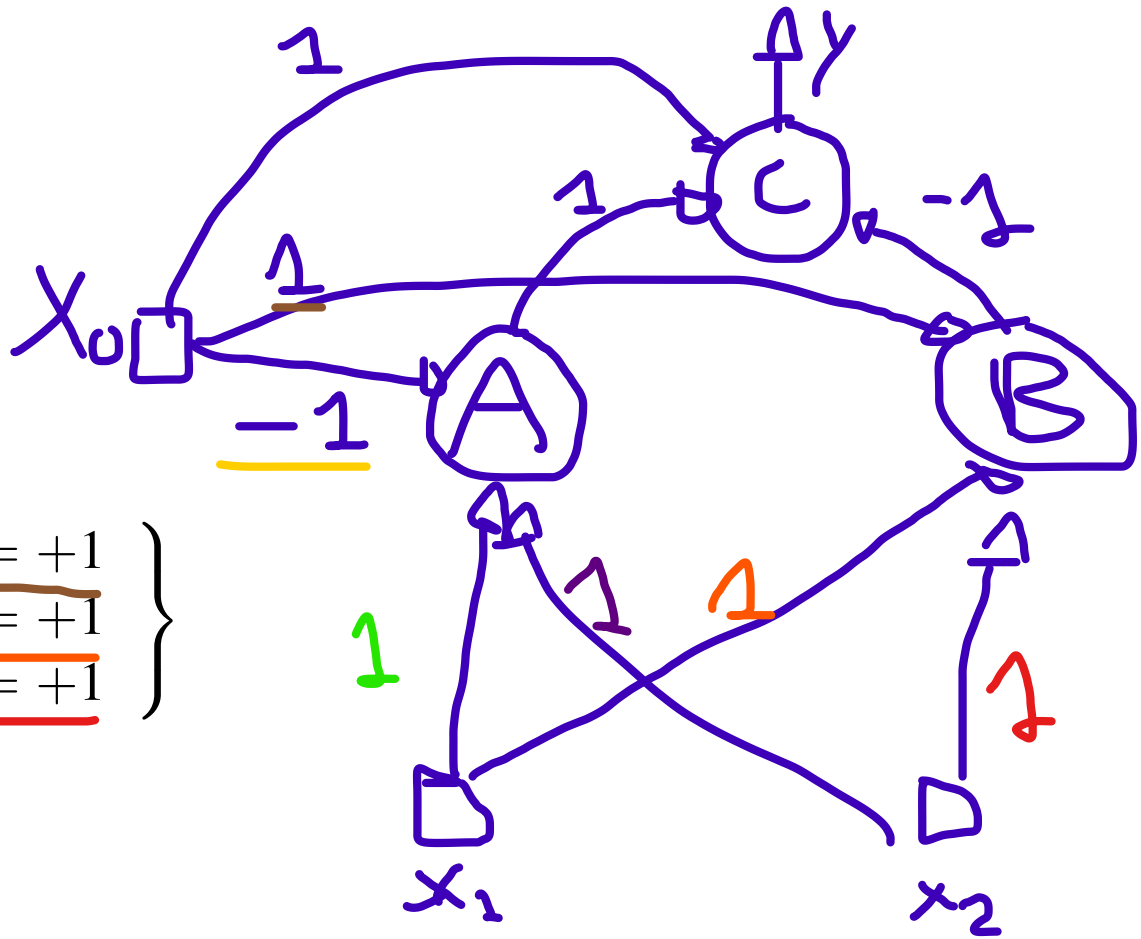
$$\rightarrow \left\{ \begin{array}{l} w_{C0} = +1 \\ w_{C1} = -1 \\ w_{C2} = +1 \end{array} \right\} \rightarrow y_C = \text{sgn}(y_A - y_B - 1)$$

toma la salida del perceptron a, le resta la salida del perceptron b y le resta -1

# ¿Cómo es la arquitectura de esta red neuronal?

$$\left\{ \begin{array}{l} w_{C0} = +1 \\ w_{C1} = -1 \\ w_{C2} = +1 \end{array} \right\}$$

$$\left\{ \begin{array}{l} w_{A0} = -1 \\ w_{A1} = +1 \\ w_{A2} = +1 \end{array} \right\} \quad \left\{ \begin{array}{l} w_{B0} = +1 \\ w_{B1} = +1 \\ w_{B2} = +1 \end{array} \right\}$$



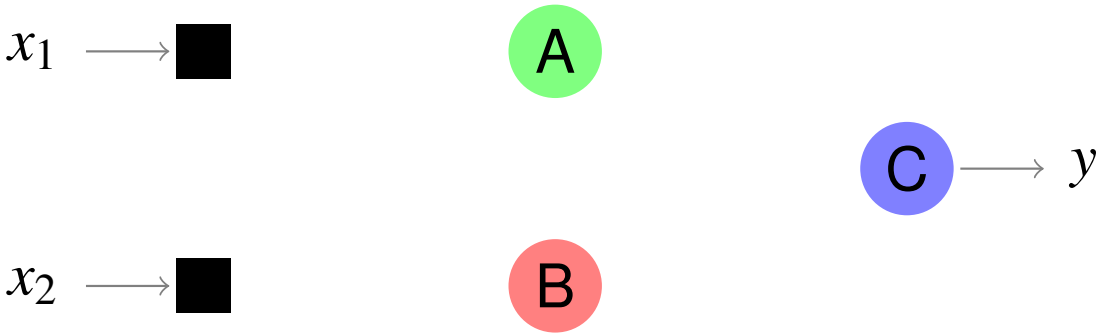

---

$$\left. \begin{array}{l} y_A = \text{sgn}(x_2 + x_1 + 1) \\ y_B = \text{sgn}(x_2 + x_1 - 1) \end{array} \right\} \rightarrow y_C = \text{sgn}(y_A - y_B - 1)$$

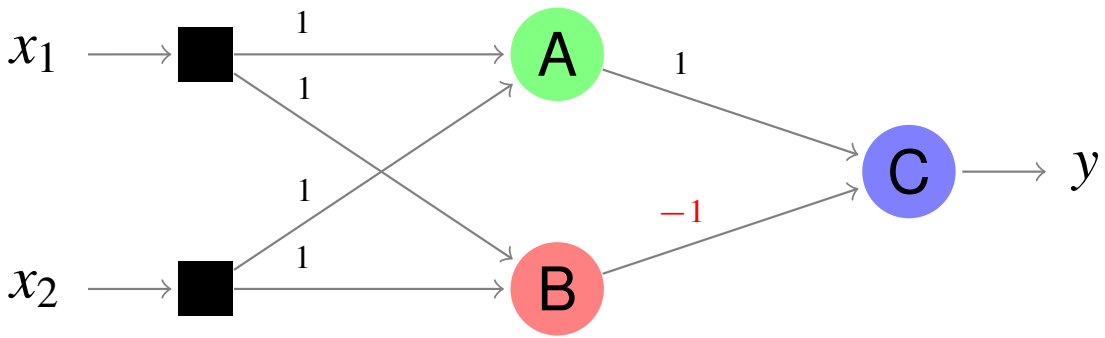
x0 * 1 o -1 para el ultimo termino de yA yB

# Y... ¿resolverá el XOR?

# Y... ¿resolverá el XOR?

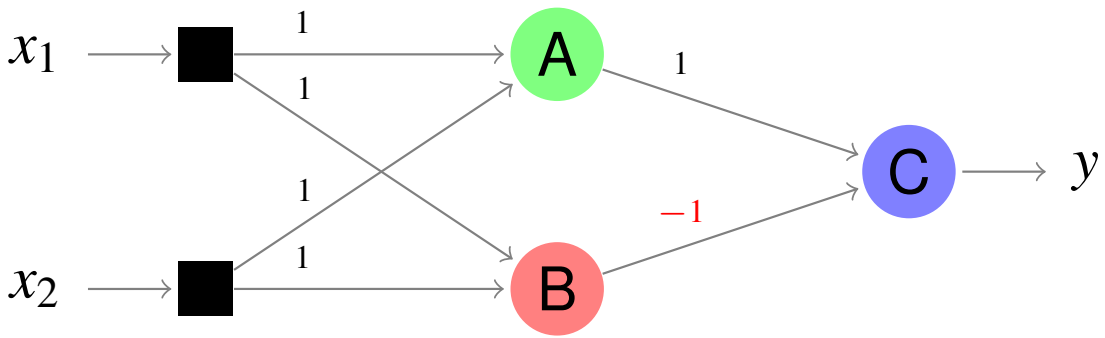


# Y... ¿resolverá el XOR?

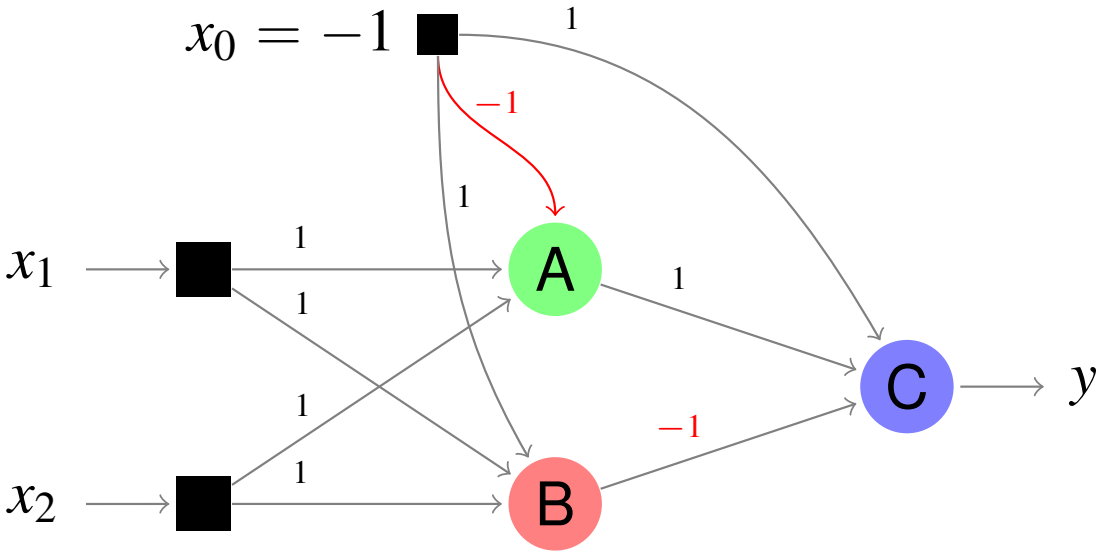


# Y... ¿resolverá el XOR?

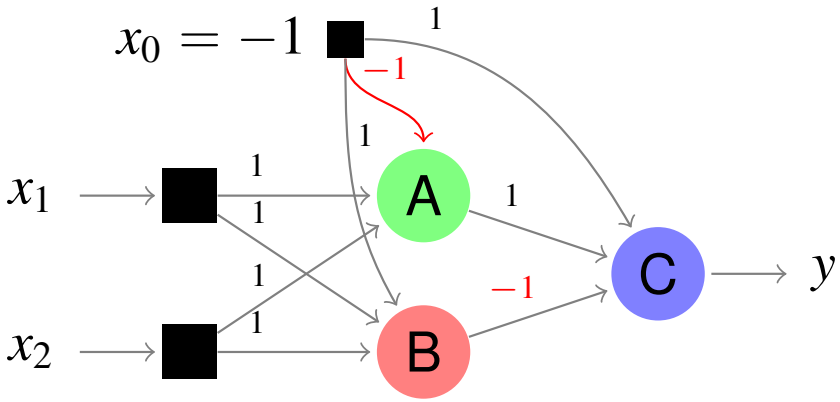
$$x_0 = -1$$



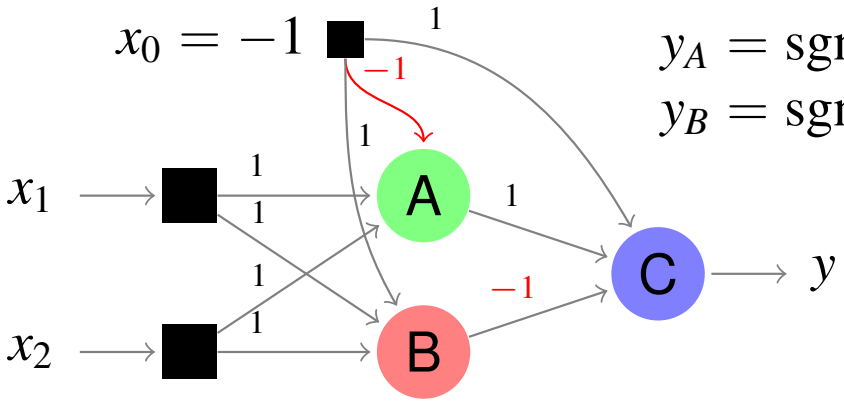
# Y... ¿resolverá el XOR?



# Y... ¿resolverá el XOR?



# Y... ¿resolverá el XOR?




$$\left.\begin{aligned} y_A &= \operatorname{sgn}(x_2 + x_1 + 1) \\ y_B &= \operatorname{sgn}(x_2 + x_1 - 1) \end{aligned}\right\} \rightarrow y_C = \operatorname{sgn}(y_A - y_B - 1)$$

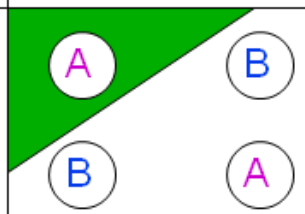
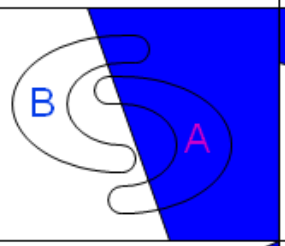
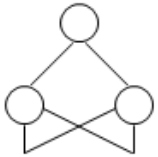
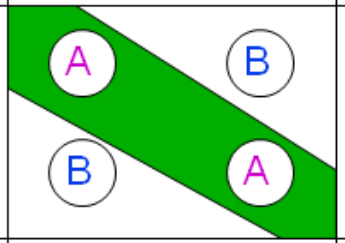
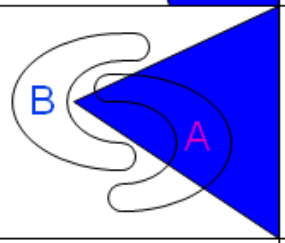
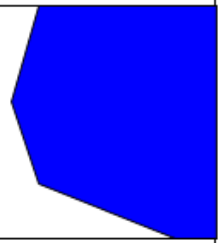
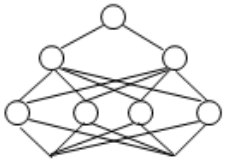
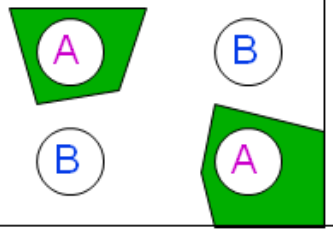
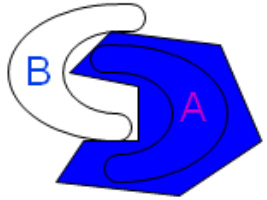
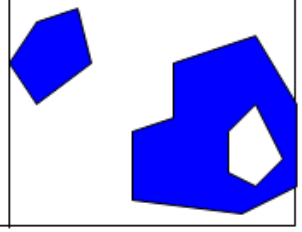
# Perceptrón multicapa: regiones de decisión y arquitectura

Diego Milone

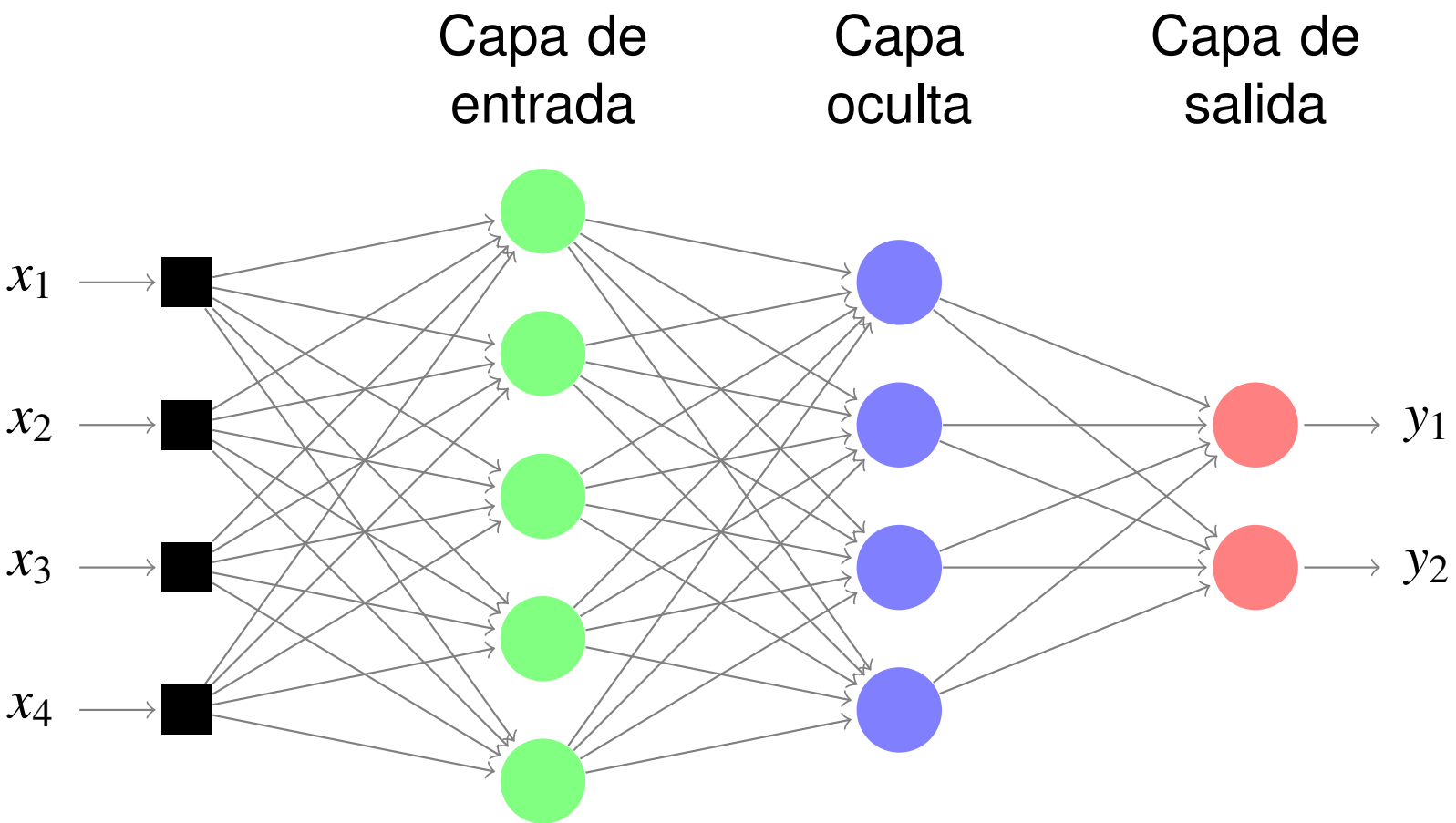
Inteligencia Computacional
Departamento de Informática

FICH-UNL

# Regiones de decisión

| *Estructura* | *Tipos de regiones de decisión* | *Problema XOR* | *Separación en clases* | *Formas regiones más generales* |
|---|---|---|---|---|
| *Una capa*  | *hemiplano limitado por hiperplano* |   |   |  |
| *Dos capas*  | *Regiones convexas abiertas o cerradas* |   |   |  |
| *Tres capas*  | Arbitrarias (Complejidad limitada por N°. de Nodos) |   |   |  |

# Arquitectura del perceptrón multicapa



# Arquitectura del perceptrón multicapa

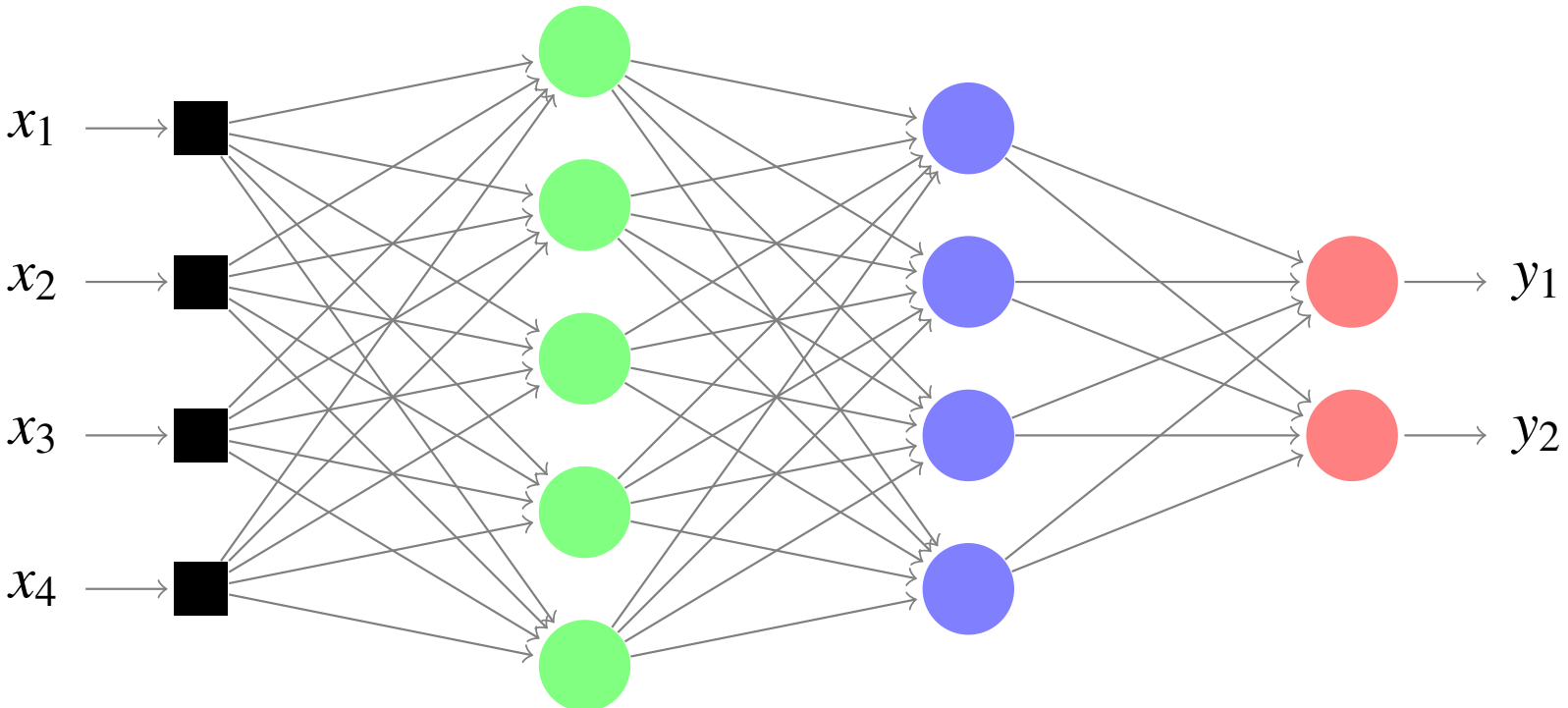
matriz de entrada donde I denota que es la capa 1 Contiene los pesos de x1 x2 x3 y x4 es decir 4 * 5 nodos =20

pesos que unen a la primer con la segunda capa de nueronas

$\mathbf{w}^{I}$ $\mathbf{w}^{II}$ $\mathbf{w}^{III}$

$\mathbf{y}^{I}$ $\mathbf{y}^{II}$ $\mathbf{y}^{III}$

salidas de cada neurona



# Cálculo de las salidas en cada capa

- Capa I:

$$v_j^I = \left\langle \mathbf{w}_j^I, \mathbf{x} \right\rangle = \sum_{i=0}^{N} w_{ji}^I x_i \quad \text{(completo } \mathbf{v}^I = \mathbf{W}^I \mathbf{x}\text{)}$$

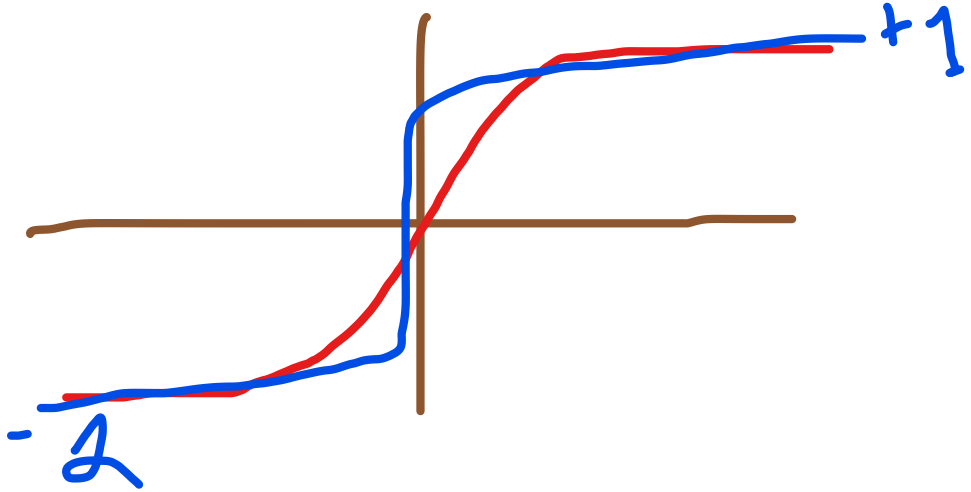
# Cálculo de las salidas en cada capa

- Capa I:

$$v_j^I = \left\langle \mathbf{w}_j^I, \mathbf{x} \right\rangle = \sum_{i=0}^{N} w_{ji}^I x_i \quad \text{(completo } \mathbf{v}^I = \mathbf{W}^I \mathbf{x}\text{)}$$

$$y_j^I = \phi(v_j^I) = \frac{2}{1 + e^{-b v_j^I}} - 1 \quad \text{(simétrica } \pm 1\text{)}$$

con el b podemos transformar la funcion sigmodea a la funcion signo sin entrar en discontinuidades



# Cálculo de las salidas en cada capa

- Capa I:

$$v_j^I = \left\langle \mathbf{w}_j^I, \mathbf{x} \right\rangle = \sum_{i=0}^{N} w_{ji}^I x_i \quad \text{(completo } \mathbf{v}^I = \mathbf{W}^I \mathbf{x}\text{)}$$

$$y_j^I = \phi(v_j^I) = \frac{2}{1 + e^{-b v_j^I}} - 1 \quad \text{(simétrica } \pm 1\text{)}$$

- Capa II:

$$v_j^{II} = \left\langle \mathbf{w}_j^{II}, \mathbf{y}^I \right\rangle \quad \rightarrow \quad y_j^{II} = \phi(v_j^{II})$$

- Capa III:

$$v_j^{III} = \left\langle \mathbf{w}_j^{III}, \mathbf{y}^{II} \right\rangle \quad \rightarrow \quad y_j^{III} = \phi(v_j^{III}) = y_j$$

# Propagación hacia atrás: caso general y capa de salida

Diego Milone

Inteligencia Computacional
Departamento de Informática

FICH-UNL

# Criterio de error

Suma del error cuadrático instantáneo

$$\xi(n) = \frac{1}{2} \sum_{j=1}^{M} e_j^2(n)$$

# Aplicación del gradiente (caso general)

$$\Delta w_{ji}(n) = -\mu \frac{\partial \xi(n)}{\partial w_{ji}(n)}$$

# Aplicación del gradiente (caso general)

$$\boxed{\Delta w_{ji}(n) = -\mu \frac{\partial \xi(n)}{\partial w_{ji}(n)}}$$

$$\frac{\partial \xi(n)}{\partial w_{ji}(n)} = \frac{\partial \xi(n)}{\partial e_j(n)} \frac{\partial e_j(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)} \frac{\partial v_j(n)}{\partial w_{ji}(n)}$$

# Aplicación del gradiente (caso general)

$$\boxed{\Delta w_{ji}(n) = -\mu \frac{\partial \xi(n)}{\partial w_{ji}(n)}}$$

$$\frac{\partial \xi(n)}{\partial w_{ji}(n)} = \frac{\partial \xi(n)}{\partial e_j(n)} \frac{\partial e_j(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)} \boxed{\frac{\partial v_j(n)}{\partial w_{ji}(n)}}$$

$$\frac{\partial v_j(n)}{\partial w_{ji}(n)} = \frac{\partial \sum_{i=0}^{N} w_{ji}(n) y_i(n)}{\partial w_{ji}(n)} = y_i(n)$$

y_i : entrada a la neurona que fue pesada con el peso w_i
y_j: salida de la neurona no lineal

# Aplicación del gradiente (caso general)

$$\boxed{\Delta w_{ji}(n) = -\mu \frac{\partial \xi(n)}{\partial w_{ji}(n)}}$$

$$\frac{\partial \xi(n)}{\partial w_{ji}(n)} = \boxed{\frac{\partial \xi(n)}{\partial e_j(n)} \frac{\partial e_j(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)}} y_i(n)$$

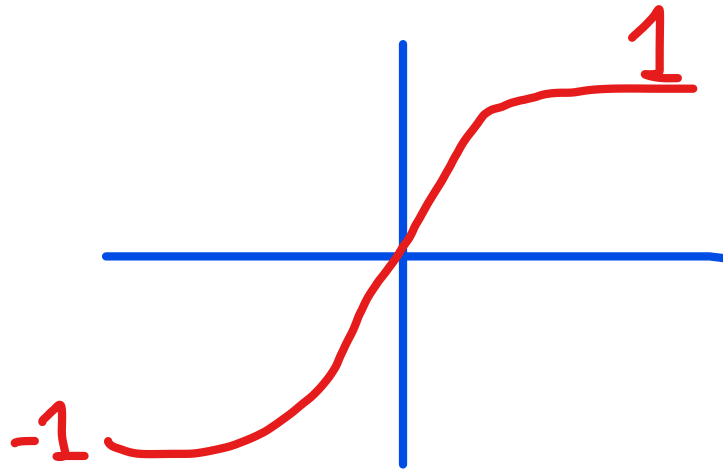
Gradiente de error local instantáneo: $\delta_j = \frac{\partial \xi(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)}$

# Aplicación del gradiente (caso general)

$$\boxed{\Delta w_{ji}(n) = \mu \delta_j(n) y_i(n)}$$

Gradiente de error local instantáneo: $\delta_j = \dfrac{\partial \xi(n)}{\partial y_j(n)} \boxed{\dfrac{\partial y_j(n)}{\partial v_j(n)}}$

# Derivada de la función de activación simétrica (1/2)




$$
\begin{aligned}
\frac{\partial y_j(n)}{\partial v_j(n)} &= \frac{\partial \left\{ \frac{2}{1+e^{-v_j(n)}} - 1 \right\}}{\partial v_j(n)} \\
&= 2 \frac{e^{-v_j(n)}}{\left(1 + e^{-v_j(n)}\right)^2} \\
&= 2 \frac{1}{1 + e^{-v_j(n)}} \frac{e^{-v_j(n)}}{1 + e^{-v_j(n)}} \\
&= 2 \frac{1}{1 + e^{-v_j(n)}} \frac{\overbrace{-1+1}^{0} + e^{-v_j(n)}}{1 + e^{-v_j(n)}} \\
&= 2 \frac{1}{1 + e^{-v_j(n)}} \left( \frac{-1}{1 + e^{-v_j(n)}} + \frac{1 + e^{-v_j(n)}}{1 + e^{-v_j(n)}} \right)
\end{aligned}
$$

# Derivada de la función de activación simétrica (2/2)

$$
\begin{aligned}
\frac{\partial y_j(n)}{\partial v_j(n)} &= 2\frac{1}{1+e^{-v_j(n)}}\left(1-\frac{1}{1+e^{-v_j(n)}}\right) \\
&= 2\frac{y_j(n)+1}{2}\left(1-\frac{y_j(n)+1}{2}\right) \\
&= (y_j(n)+1)\left(1-\frac{y_j(n)+1}{2}\right) \\
&= (y_j(n)+1)\left(\frac{2-y_j(n)-1}{2}\right) \\
&= \frac{1}{2}(y_j(n)+1)(y_j(n)-1)
\end{aligned}
$$

derivada de la simbodia = 1/2 * la salida +1 * la salida - 1

# Aplicación del gradiente (caso general)

$$\boxed{\Delta w_{ji}(n) = \mu \delta_j(n) y_i(n)}$$

Gradiente de error local instantáneo: $\delta_j = -\frac{\partial \xi(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)}$

$$\delta_j = \frac{\partial \xi(n)}{\partial y_j(n)} \frac{1}{2}(1 + y_j(n))(1 - y_j(n))$$

# Retropropagación en la capa III (salida)

$$\Delta w_{ji}^{III}(n) = \mu \delta_j^{III}(n) y_i^{II}(n)$$

gradiente de error local para la capa 3

velocidad de aprendizaje

entrada a la capa 3, salida de la capa 2

# Retropropagación en la capa III (salida)

$$\Delta w_{ji}^{III}(n) = \mu \delta_j^{III}(n) y_i^{II}(n)$$

$$\delta_j^{III}(n) = -\frac{\partial \xi(n)}{\partial y_j^{III}(n)} \frac{1}{2}(1 + y_j^{III}(n))(1 - y_j^{III}(n))$$

# Retropropagación en la capa III (salida)

$$\Delta w_{ji}^{III}(n) = \mu \delta_j^{III}(n) y_i^{II}(n)$$

$$\delta_j^{III}(n) = -\frac{\partial \xi(n)}{\partial y_j^{III}(n)} \frac{1}{2}(1 + y_j^{III}(n))(1 - y_j^{III}(n))$$

$$\delta_j^{III}(n) = -\frac{\partial \xi(n)}{\partial e_j(n)} \frac{\partial e_j(n)}{\partial y_j^{III}(n)} \frac{1}{2}(1 + y_j^{III}(n))(1 - y_j^{III}(n))$$

# Retropropagación en la capa III (salida)

$$\delta_j^{III}(n) = -\frac{\partial\left\{\frac{1}{2}\sum_j e_j^2(n)\right\}}{\partial e_j(n)} \cdot \frac{\partial\left\{d_j^{III}(n) - y_j^{III}(n)\right\}}{\partial y_j^{III}(n)} \cdot$$
$$\cdot\frac{1}{2}(1 + y_j^{III}(n))(1 - y_j^{III}(n))$$

# Retropropagación en la capa III (salida)

$$\delta_j^{III}(n) = -\frac{\partial\left\{\frac{1}{2}\sum_j e_j^2(n)\right\}}{\partial e_j(n)} \cdot \frac{\partial\left\{d_j^{III}(n) - y_j^{III}(n)\right\}}{\partial y_j^{III}(n)} \cdot$$

$$\cdot\frac{1}{2}(1 + y_j^{III}(n))(1 - y_j^{III}(n))$$

$$\delta_j^{III}(n) = \tfrac{1}{2}e_j(n)(1 + y_j^{III}(n))(1 - y_j^{III}(n))\bigstar$$

Derivada de la funcion de activacion

error (solo se puede medir en la capa de salida)
ya que no hay salidas deseadas para las capas ocultas

# Retropropagación en la capa III (salida)

$$\delta_j^{III}(n) = -\frac{\partial\left\{\frac{1}{2}\sum_j e_j^2(n)\right\}}{\partial e_j(n)} \cdot \frac{\partial\left\{d_j^{III}(n) - y_j^{III}(n)\right\}}{\partial y_j^{III}(n)} \cdot$$
$$\cdot\frac{1}{2}(1 + y_j^{III}(n))(1 - y_j^{III}(n))$$

$$\delta_j^{III}(n) = \tfrac{1}{2}e_j(n)(1 + y_j^{III}(n))(1 - y_j^{III}(n))\bigstar$$

$$\boxed{\Delta w_{ji}^{III}(n) = \eta e_j(n)(1 + y_j^{III}(n))(1 - y_j^{III}(n))y_i^{II}(n)}$$

# Propagación hacia atrás: capas ocultas

Diego Milone

Inteligencia Computacional
Departamento de Informática

FICH-UNL

# Retropropagación en la capa II (oculta)

$$\Delta w_{ji}^{II}(n) = \mu \delta_j^{II}(n) y_i^I(n)$$

# Retropropagación en la capa II (oculta)

$$\Delta w_{ji}^{II}(n) = \mu \delta_j^{II}(n) y_i^I(n)$$

$$\delta_j^{II}(n) = -\frac{\partial \xi(n)}{\partial y_j^{II}(n)} \frac{1}{2}(1 + y_j^{II}(n))(1 - y_j^{II}(n))$$

# Retropropagación en la capa II (oculta)

$$\Delta w_{ji}^{II}(n) = \mu \delta_j^{II}(n) y_i^I(n)$$

$$\delta_j^{II}(n) = -\frac{\partial \xi(n)}{\partial y_j^{II}(n)} \frac{1}{2}(1 + y_j^{II}(n))(1 - y_j^{II}(n))$$

$$\delta_j^{II}(n) = -\frac{\partial \left\{ \frac{1}{2} \sum_k e_k^2(n) \right\}}{\partial y_j^{II}(n)} \frac{1}{2}(1 + y_j^{II}(n))(1 - y_j^{II}(n))$$

# Retropropagación en la capa II (oculta)

$$\Delta w_{ji}^{II}(n) = \mu \delta_j^{II}(n) y_i^I(n)$$

$$\delta_j^{II}(n) = -\frac{\partial \xi(n)}{\partial y_j^{II}(n)} \frac{1}{2}(1 + y_j^{II}(n))(1 - y_j^{II}(n))$$

$$\delta_j^{II}(n) = -\frac{\partial \left\{ \frac{1}{2} \sum_k e_k^2(n) \right\}}{\partial y_j^{II}(n)} \frac{1}{2}(1 + y_j^{II}(n))(1 - y_j^{II}(n))$$

$$\delta_j^{II}(n) = -\frac{1}{2} \sum_k \frac{\partial e_k^2(n)}{\partial y_j^{II}(n)} \frac{1}{2}(1 + y_j^{II}(n))(1 - y_j^{II}(n))$$

i: salida de la capa anterior, o sea la entrada de la capa actual
j: las neuronas de la capa actual
k: la capa de salida

# Retropropagación en la capa II (oculta)

$$\Delta w_{ji}^{II}(n) = \mu \delta_j^{II}(n) y_i^I(n)$$

$$\delta_j^{II}(n) = -\frac{\partial \xi(n)}{\partial y_j^{II}(n)} \frac{1}{2}(1 + y_j^{II}(n))(1 - y_j^{II}(n))$$

$$\delta_j^{II}(n) = -\frac{\partial \left\{ \frac{1}{2} \sum_k e_k^2(n) \right\}}{\partial y_j^{II}(n)} \frac{1}{2}(1 + y_j^{II}(n))(1 - y_j^{II}(n))$$

$$\delta_j^{II}(n) = -\frac{1}{2} \sum_k \frac{\partial e_k^2(n)}{\partial y_j^{II}(n)} \frac{1}{2}(1 + y_j^{II}(n))(1 - y_j^{II}(n))$$

$$\delta_j^{II}(n) = -\sum_k e_k(n) \frac{\partial e_k(n)}{\partial y_j^{II}(n)} \frac{1}{2}(1 + y_j^{II}(n))(1 - y_j^{II}(n))$$

# Retropropagación en la capa II (oculta)

$$\delta_j^{II}(n) = -\sum_k e_k(n) \frac{\partial e_k(n)}{\partial y_k^{III}(n)} \frac{\partial y_k^{III}(n)}{\partial v_k^{III}(n)} \frac{\partial v_k^{III}(n)}{\partial y_j^{II}(n)} \frac{1}{2}(1+y_j^{II}(n))(1-y_j^{II}(n))$$

# Retropropagación en la capa II (oculta)

$$\delta_j^{II}(n) = -\sum_k e_k(n) \frac{\partial e_k(n)}{\partial y_k^{III}(n)} \frac{\partial y_k^{III}(n)}{\partial v_k^{III}(n)} \frac{\partial v_k^{III}(n)}{\partial y_j^{II}(n)} \frac{1}{2}(1+y_j^{II}(n))(1-y_j^{II}(n))$$

error

$$\delta_j^{II}(n) = -\sum_k e_k(n) \cdot \frac{\partial \left\{ d_k^{III}(n) - y_k^{III}(n) \right\}}{\partial y_k^{III}(n)} \cdot \frac{1}{2}(1+y_k^{III}(n))(1-y_k^{III}(n)) \cdot$$

$$\cdot \frac{\partial \left\{ \sum_j w_{kj}^{III} y_j^{II}(n) \right\}}{\partial y_j^{II}(n)} \cdot \frac{1}{2}(1+y_j^{II}(n))(1-y_j^{II}(n))$$

# Retropropagación en la capa II (oculta)

$$\delta_j^{II}(n) = -\sum_k e_k(n) \frac{\partial e_k(n)}{\partial y_k^{III}(n)} \frac{\partial y_k^{III}(n)}{\partial v_k^{III}(n)} \frac{\partial v_k^{III}(n)}{\partial y_j^{II}(n)} \frac{1}{2}(1+y_j^{II}(n))(1-y_j^{II}(n))$$

$$\delta_j^{II}(n) = -\sum_k e_k(n) \cdot \frac{\partial \left\{ d_k^{III}(n) - y_k^{III}(n) \right\}}{\partial y_k^{III}(n)} \cdot \frac{1}{2}(1+y_k^{III}(n))(1-y_k^{III}(n)) \cdot$$

$$\cdot \frac{\partial \left\{ \sum_j w_{kj}^{III} y_j^{II}(n) \right\}}{\partial y_j^{II}(n)} \cdot \frac{1}{2}(1+y_j^{II}(n))(1-y_j^{II}(n))$$

$$\delta_j^{II}(n) = -\sum_k e_k(n) \cdot (-1) \cdot \frac{1}{2}(1+y_k^{III}(n))(1-y_k^{III}(n)) \cdot$$

$$\cdot w_{kj}^{III} \cdot \frac{1}{2}(1+y_j^{II}(n))(1-y_j^{II}(n))$$

# Retropropagación en la capa II (oculta)

$$\delta_j^{II}(n) = \sum_k e_k(n) \cdot \frac{1}{2}(1 + y_k^{III}(n))(1 - y_k^{III}(n)) \cdot w_{kj}^{III} \cdot \cdot \frac{1}{2}(1 + y_j^{II}(n))(1 - y_j^{II}(n))$$

# Retropropagación en la capa II (oculta)

$$\delta_j^{II}(n) = \sum_k e_k(n) \cdot \frac{1}{2}(1 + y_k^{III}(n))(1 - y_k^{III}(n)) \cdot w_{kj}^{III} \cdot \cdot \frac{1}{2}(1 + y_j^{II}(n))(1 - y_j^{II}(n))$$

Pero de la capa III$^\star$ sabemos que:

$$\delta_k^{III}(n) = \tfrac{1}{2} e_k(n)(1 + y_k^{III}(n))(1 - y_k^{III}(n))$$

# Retropropagación en la capa II (oculta)

$$\delta_j^{II}(n) = \sum_k e_k(n) \cdot \frac{1}{2}(1 + y_k^{III}(n))(1 - y_k^{III}(n)) \cdot w_{kj}^{III} \cdot \cdot \frac{1}{2}(1 + y_j^{II}(n))(1 - y_j^{II}(n))$$

Pero de la capa III$^\star$ sabemos que:

$$\delta_k^{III}(n) = \tfrac{1}{2} e_k(n)(1 + y_k^{III}(n))(1 - y_k^{III}(n))$$

Reemplzando:

$$\delta_j^{II}(n) = \sum_k \delta_k^{III}(n) w_{kj}^{III} \cdot \frac{1}{2}(1 + y_j^{II}(n))(1 - y_j^{II}(n))$$

paso por los pesos el gradiente de la 3 y actualizo la 2

# Retropropagación en la capa II (oculta)

Volviendo a:

$$\Delta w_{ji}^{II}(n) = \mu \delta_j^{II}(n) y_i^I(n)$$

# Retropropagación en la capa II (oculta)

Volviendo a:

$$\Delta w_{ji}^{II}(n) = \mu \delta_j^{II}(n) y_i^I(n)$$

Por lo tanto:

$$\boxed{\Delta w_{ji}^{II}(n) = \eta \left[ \sum_k \delta_k^{III} w_{kj}^{III}(n) \right] (1 + y_j^{II}(n))(1 - y_j^{II}(n)) y_i^I(n)}$$

# Generalizando para la capa "$p$"

$$\Delta w_{ji}^{II}(n) = \eta \left[ \sum_k \delta_k^{III} w_{kj}^{III}(n) \right] (1 + y_j^{II}(n))(1 - y_j^{II}(n)) y_i^{I}(n)$$

$$\Downarrow$$

$$\Delta w_{ji}^{(p)}(n) = \eta \left\langle \delta^{(p+1)}, \mathbf{w}_j^{(p+1)} \right\rangle (1 + y_j^{(p)}(n))(1 - y_j^{(p)}(n)) y_i^{(p-1)}(n)$$

ENTRADA

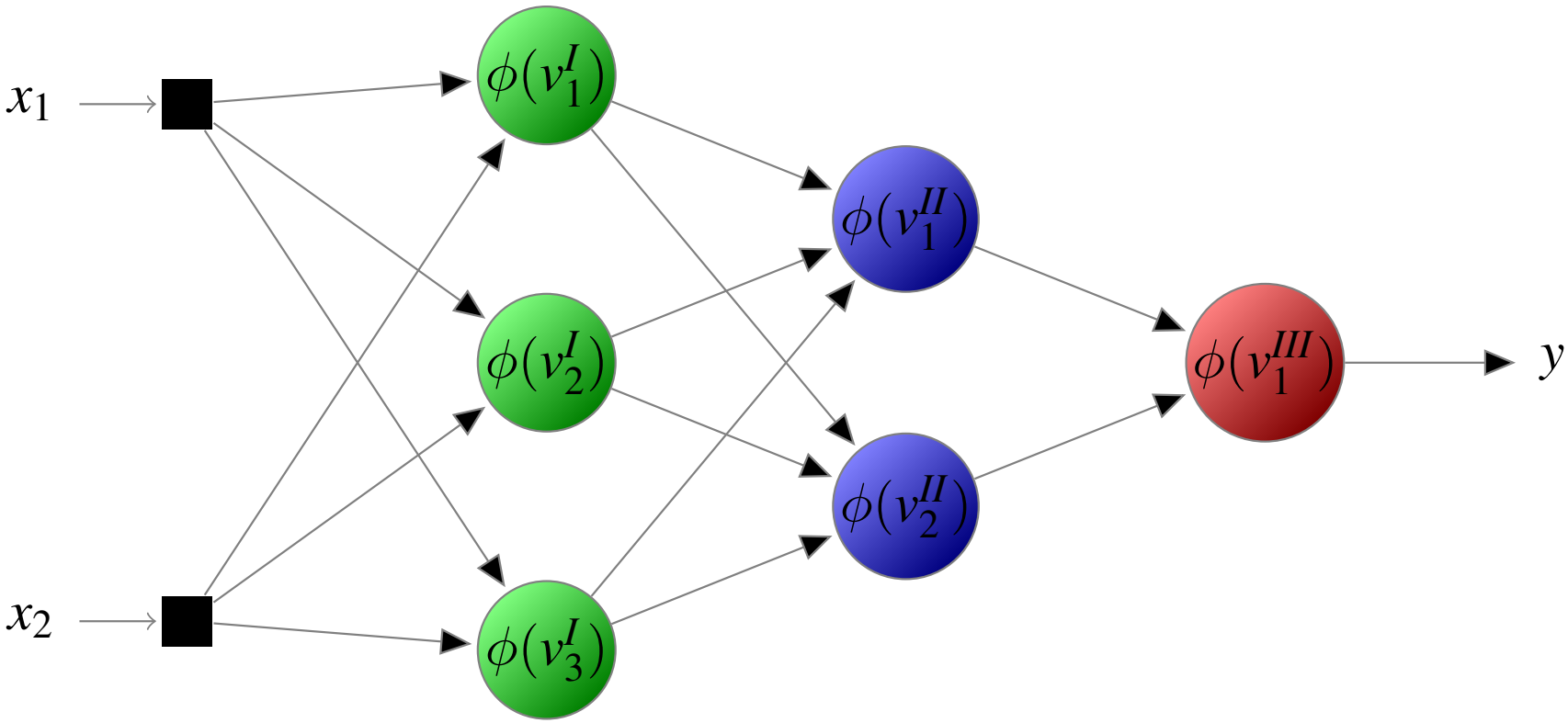
ESTE ES EL CORAZON EL TRUCO CON ESTO
PODEMOS GENERALIZAR 100 CAPAS O MAS ETC

Paso para adelante la entrada
Paso para atras el gradiente

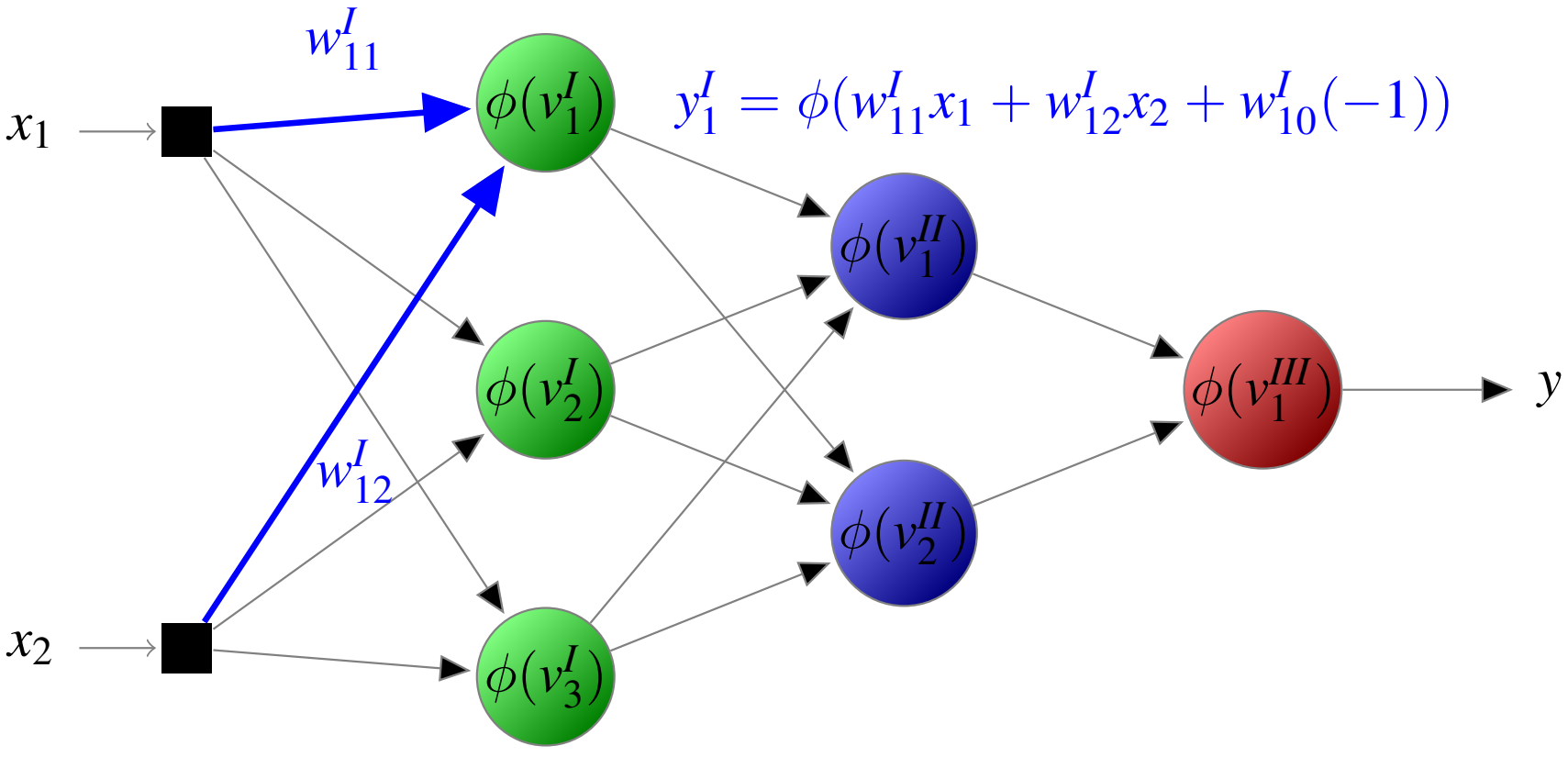
# Resumen del algoritmo de retropropagación (BP)

1. Inicialización aleatoria
2. Propagación hacia adelante
3. Propagación hacia atras
4. Adaptación de los pesos
5. Iteración: vuelve a 2 hasta convergencia o finalización

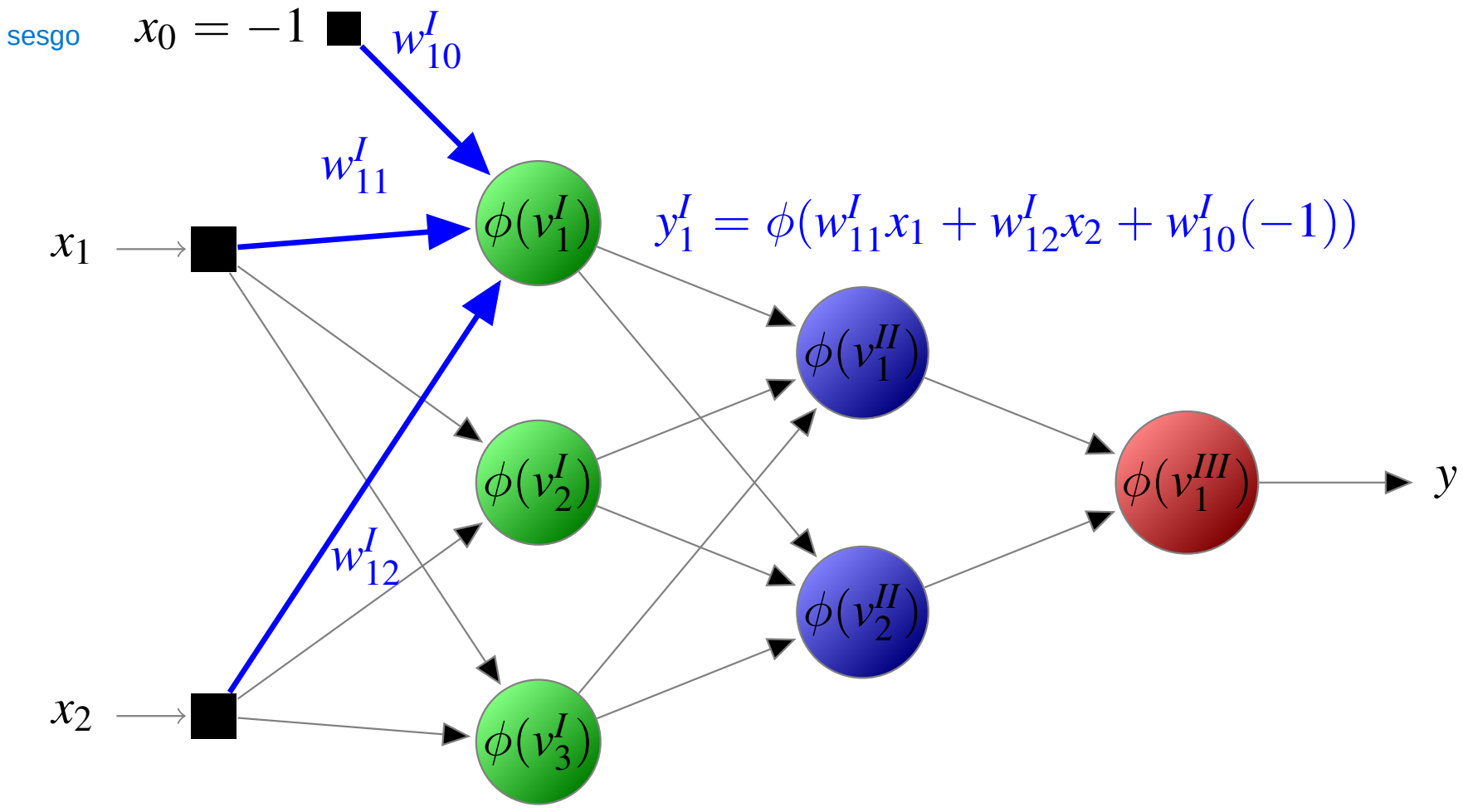
# Resumen: propagación hacia adelante



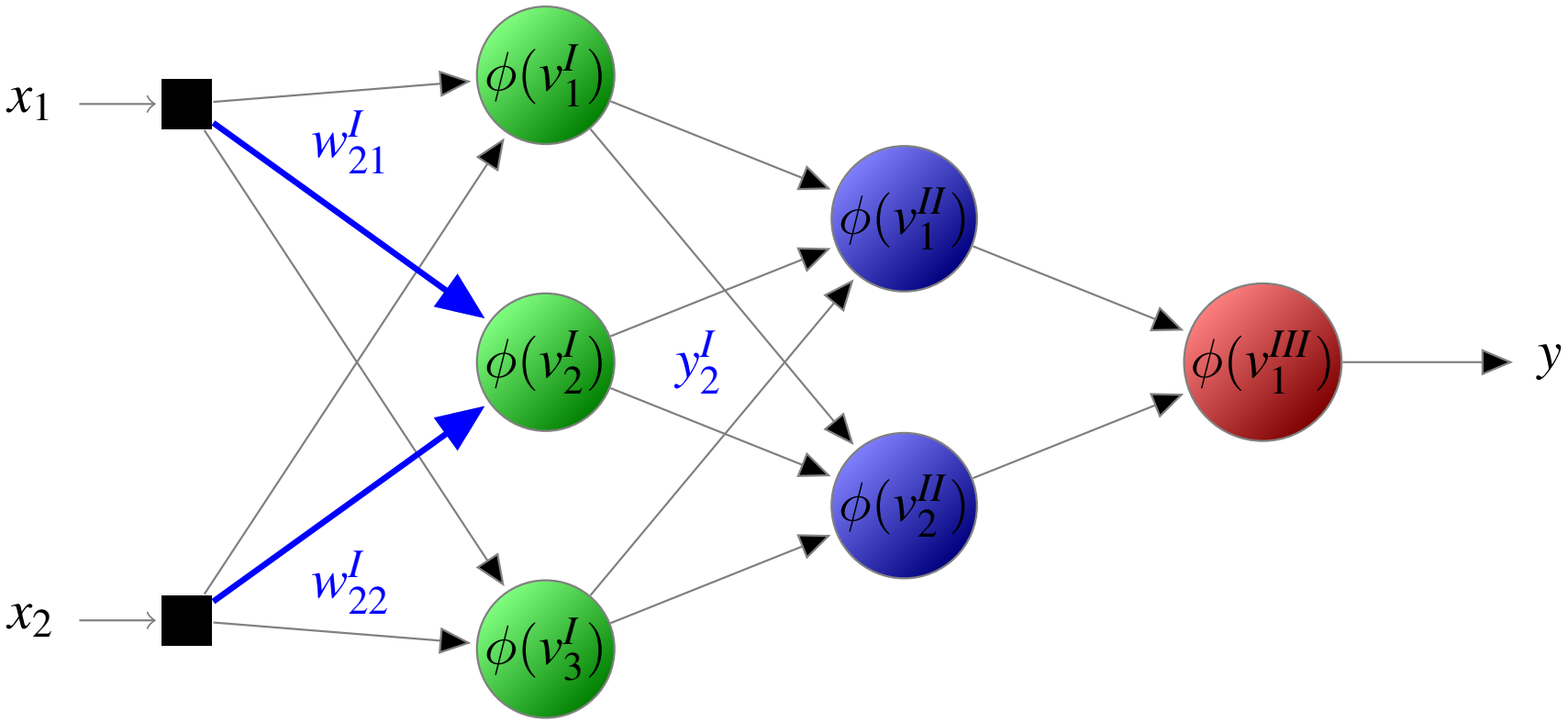
# Resumen: propagación hacia adelante



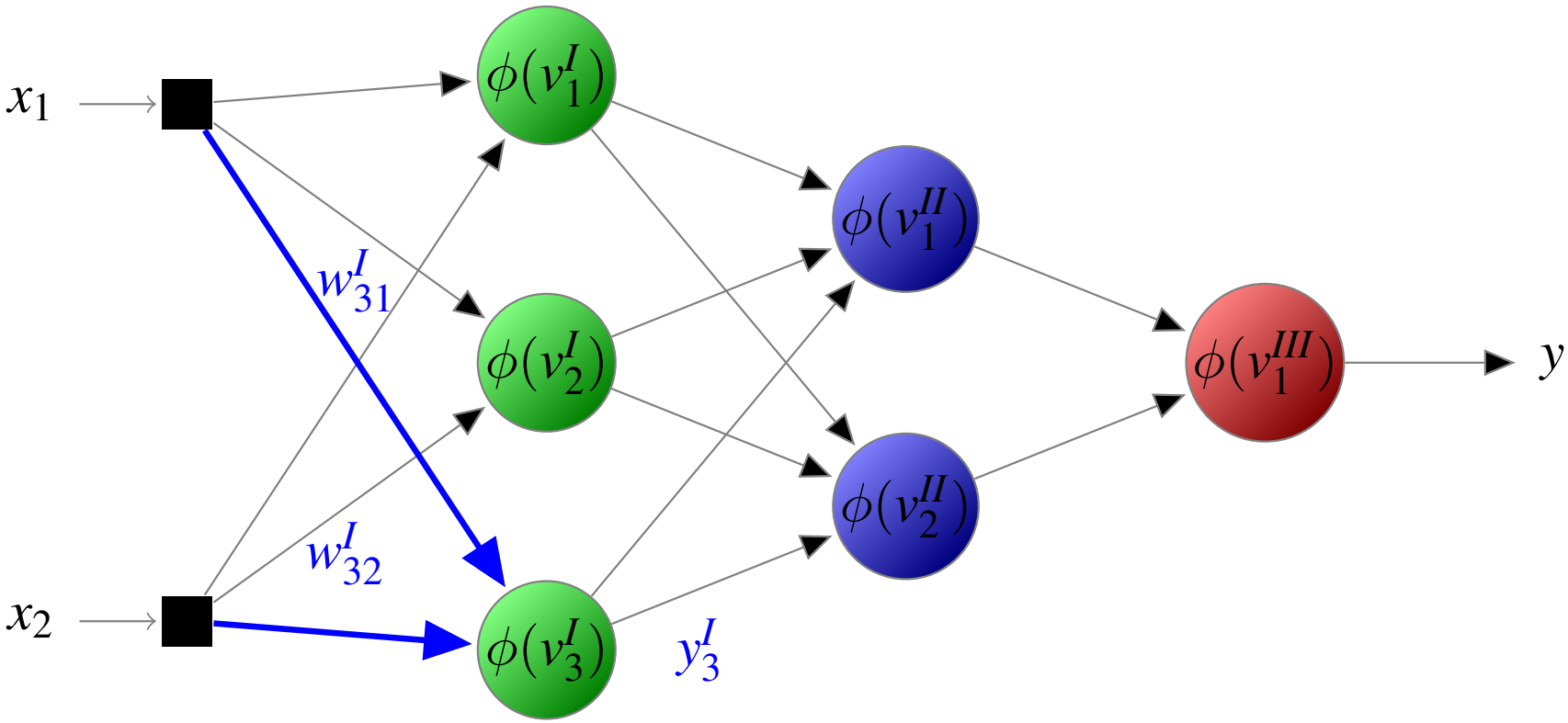
# Resumen: propagación hacia adelante



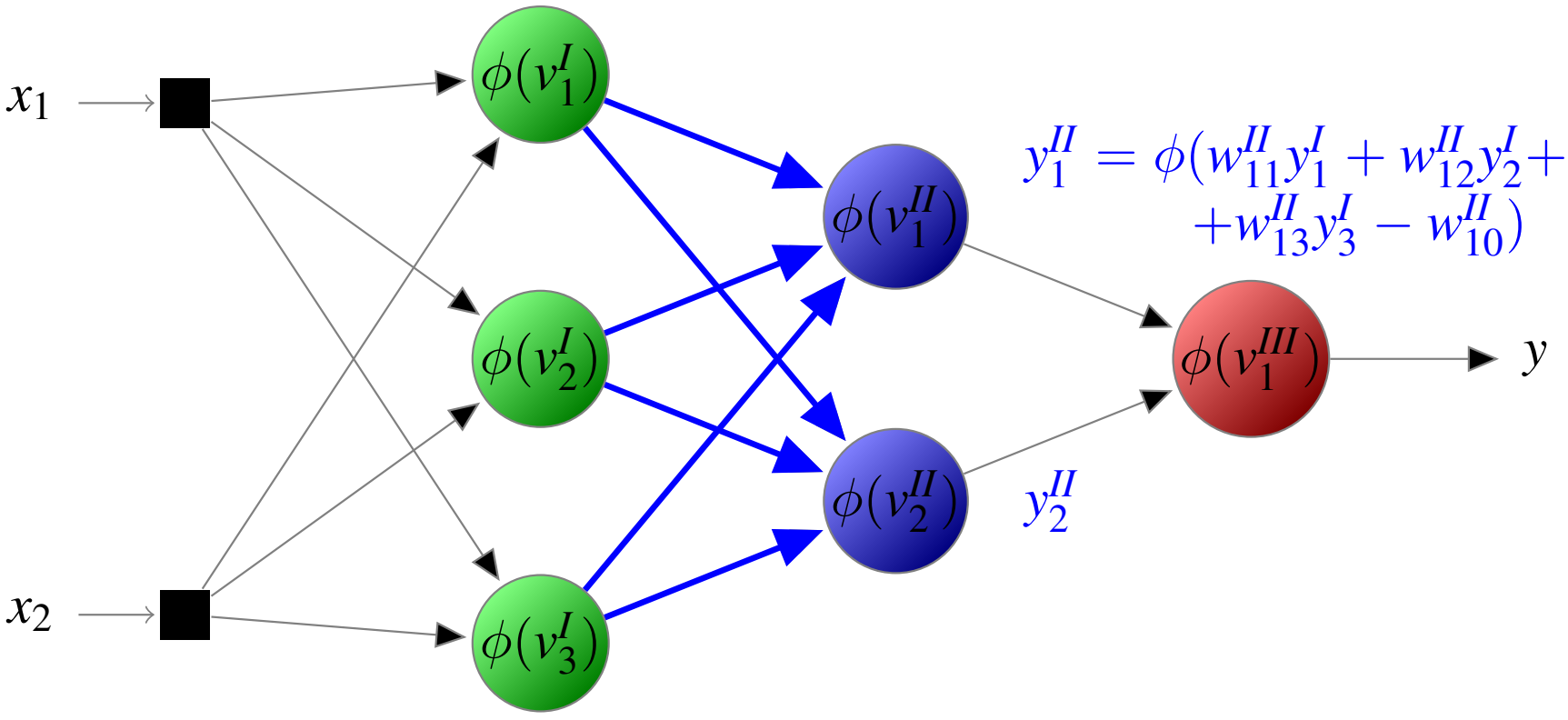
# Resumen: propagación hacia adelante



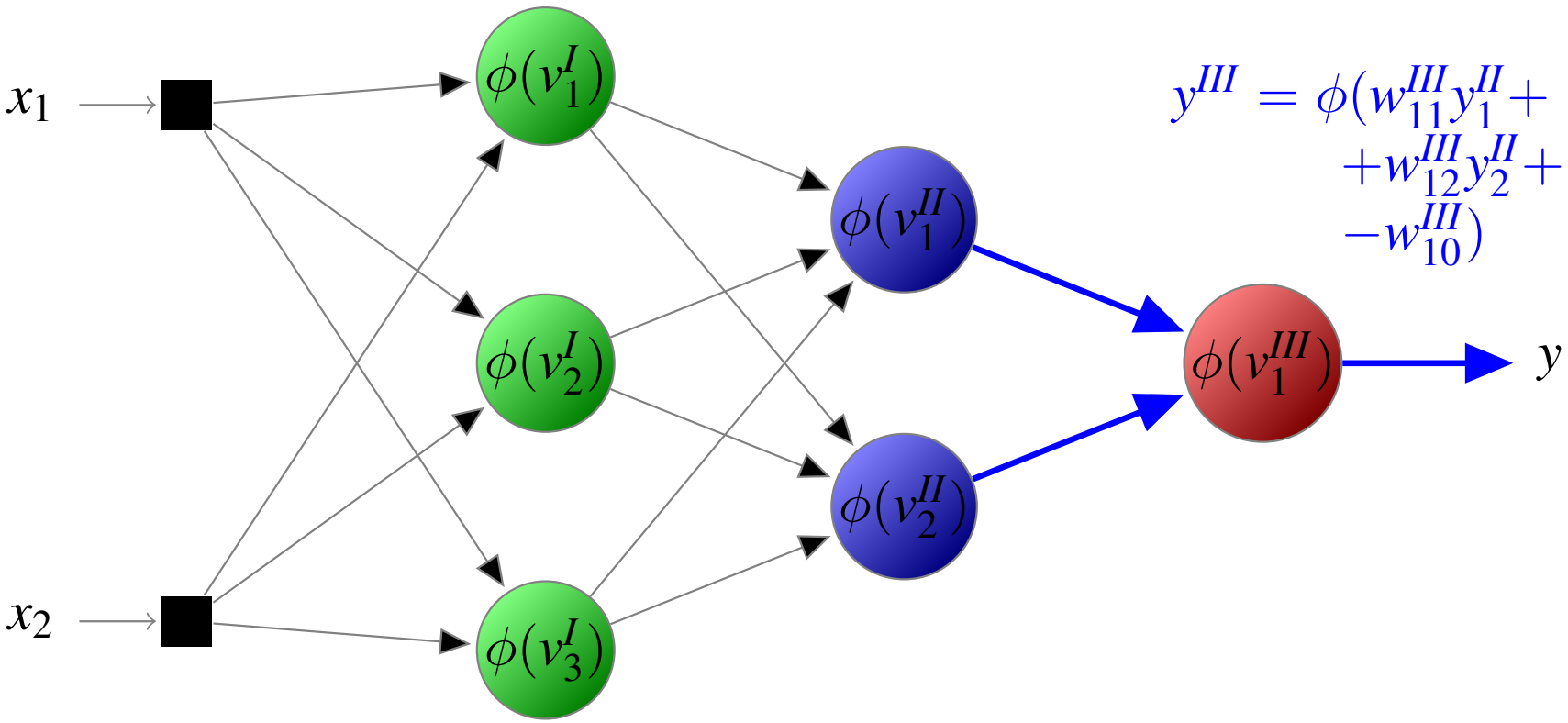
# Resumen: propagación hacia adelante



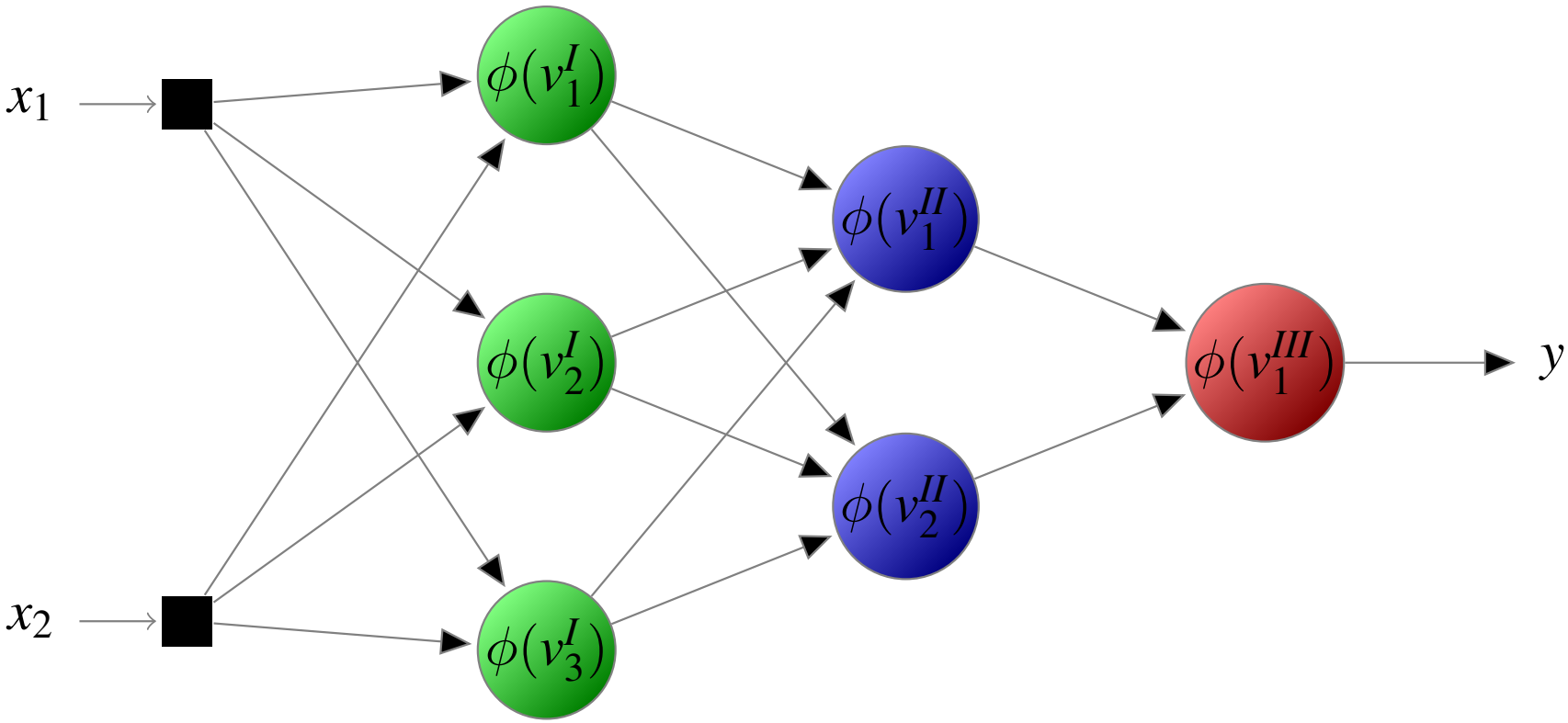
# Resumen: propagación hacia adelante



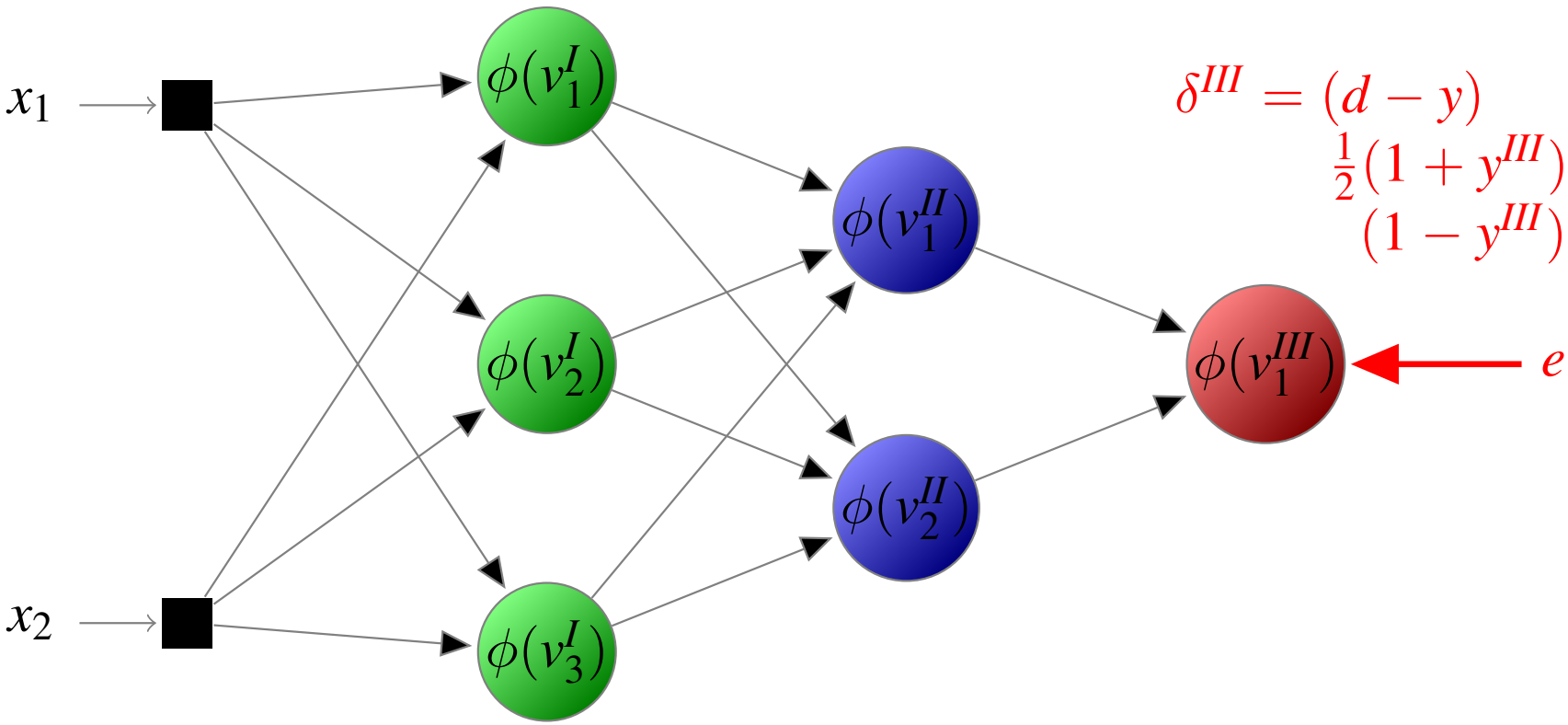
# Resumen: propagación hacia adelante



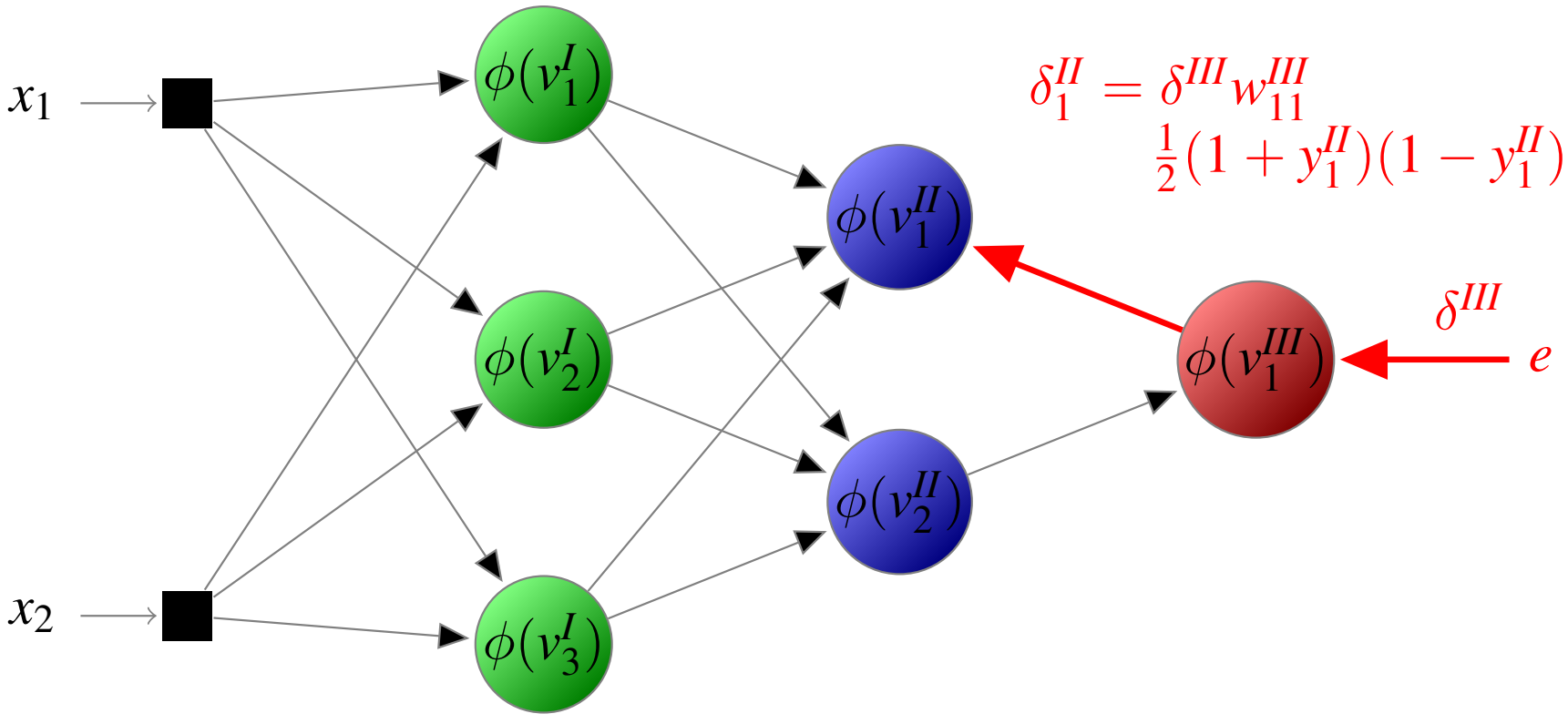
# Resumen: propagación hacia atras



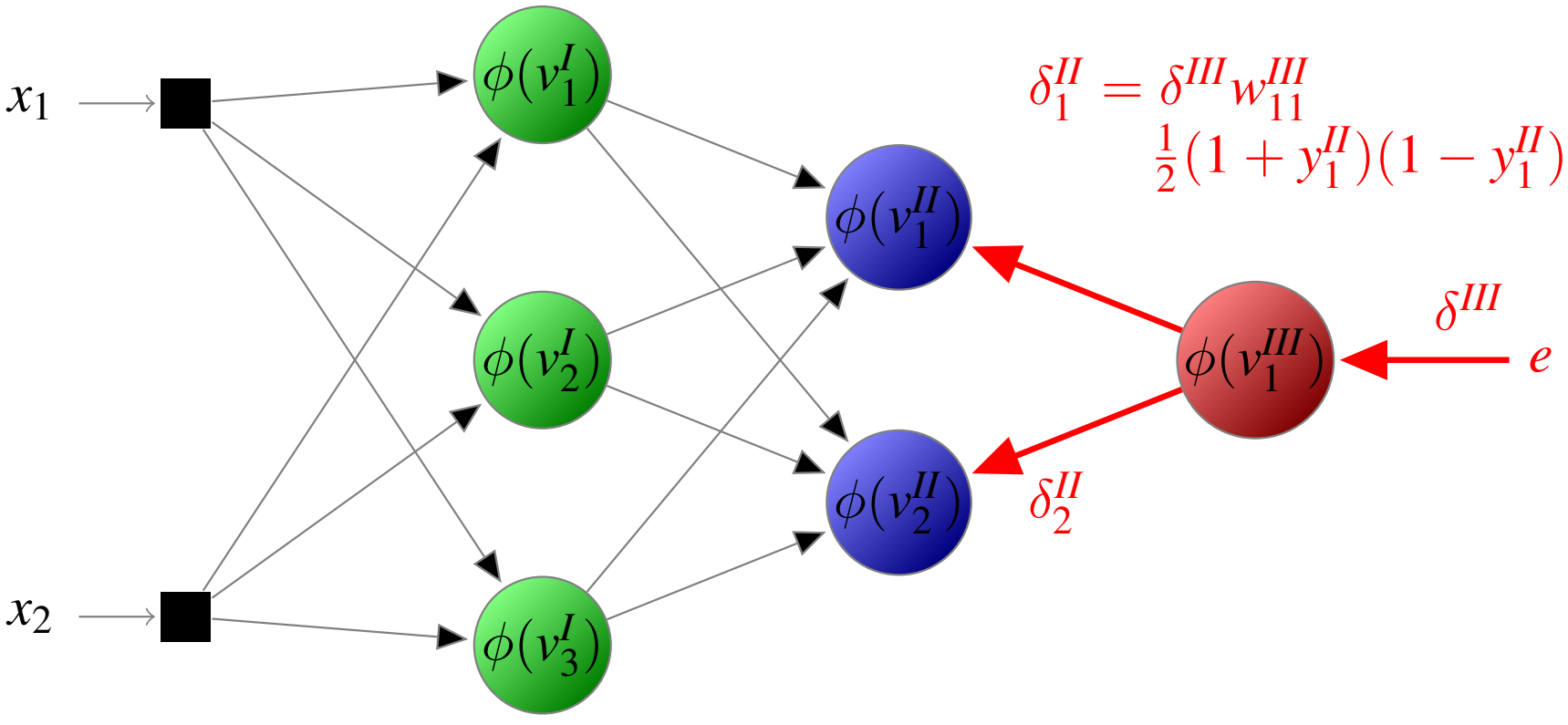
# Resumen: propagación hacia atras



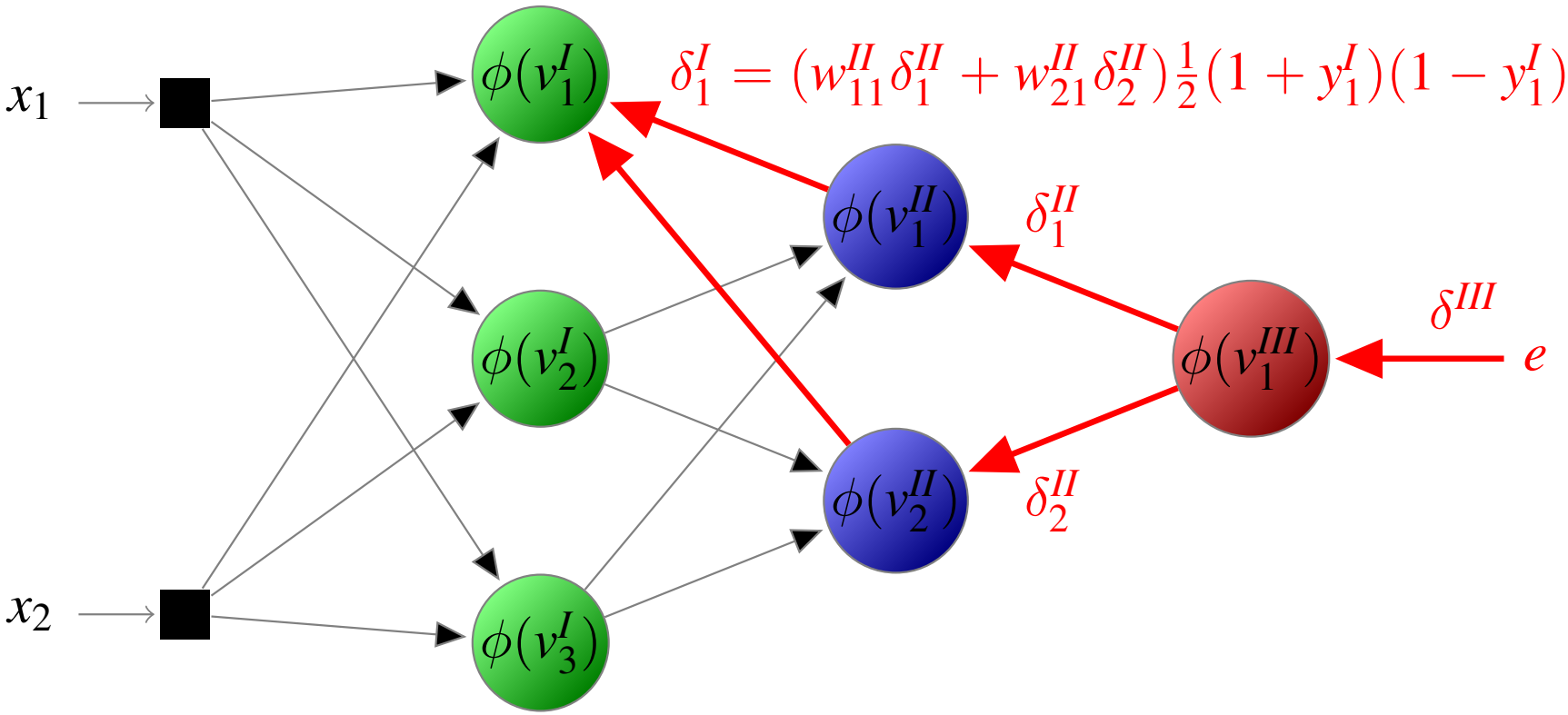
# Resumen: propagación hacia atras



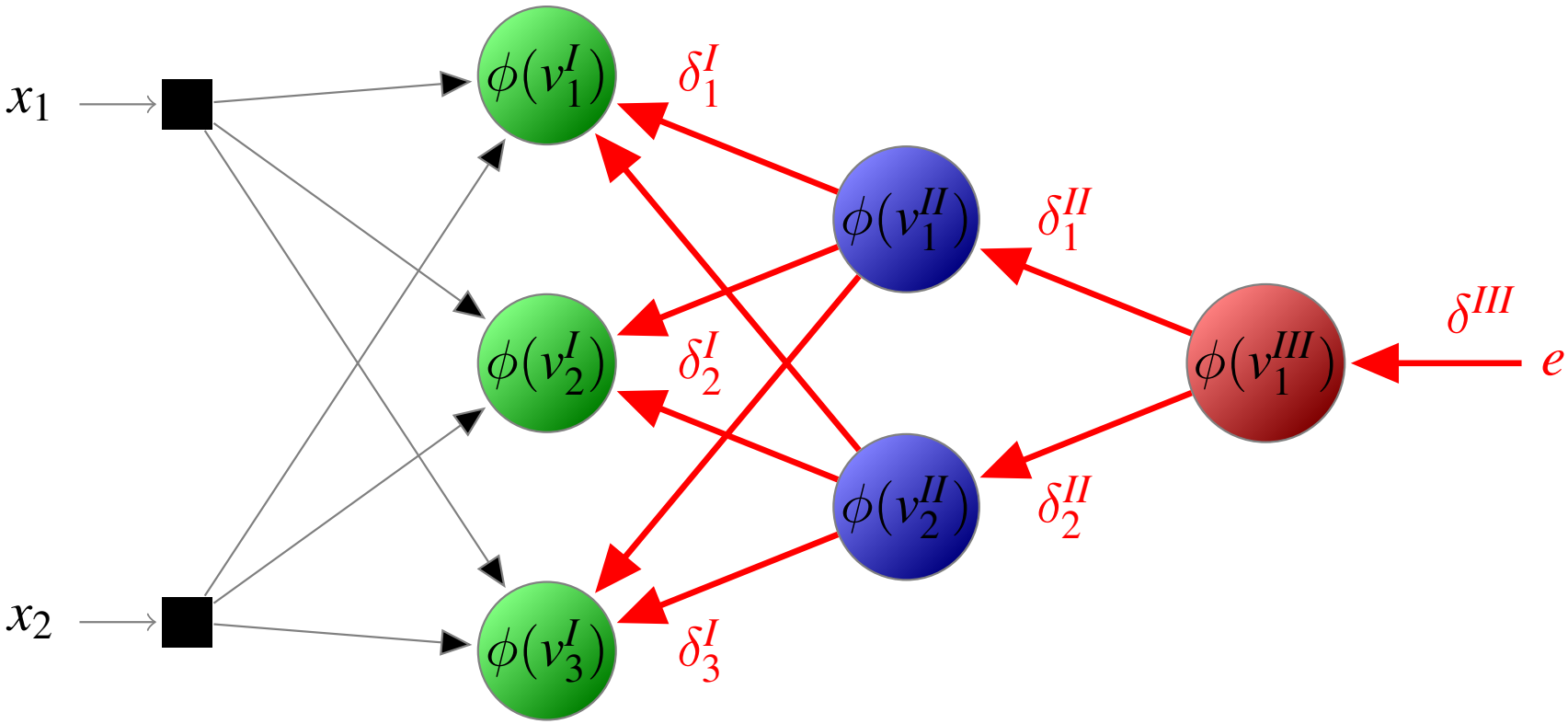
# Resumen: propagación hacia atras



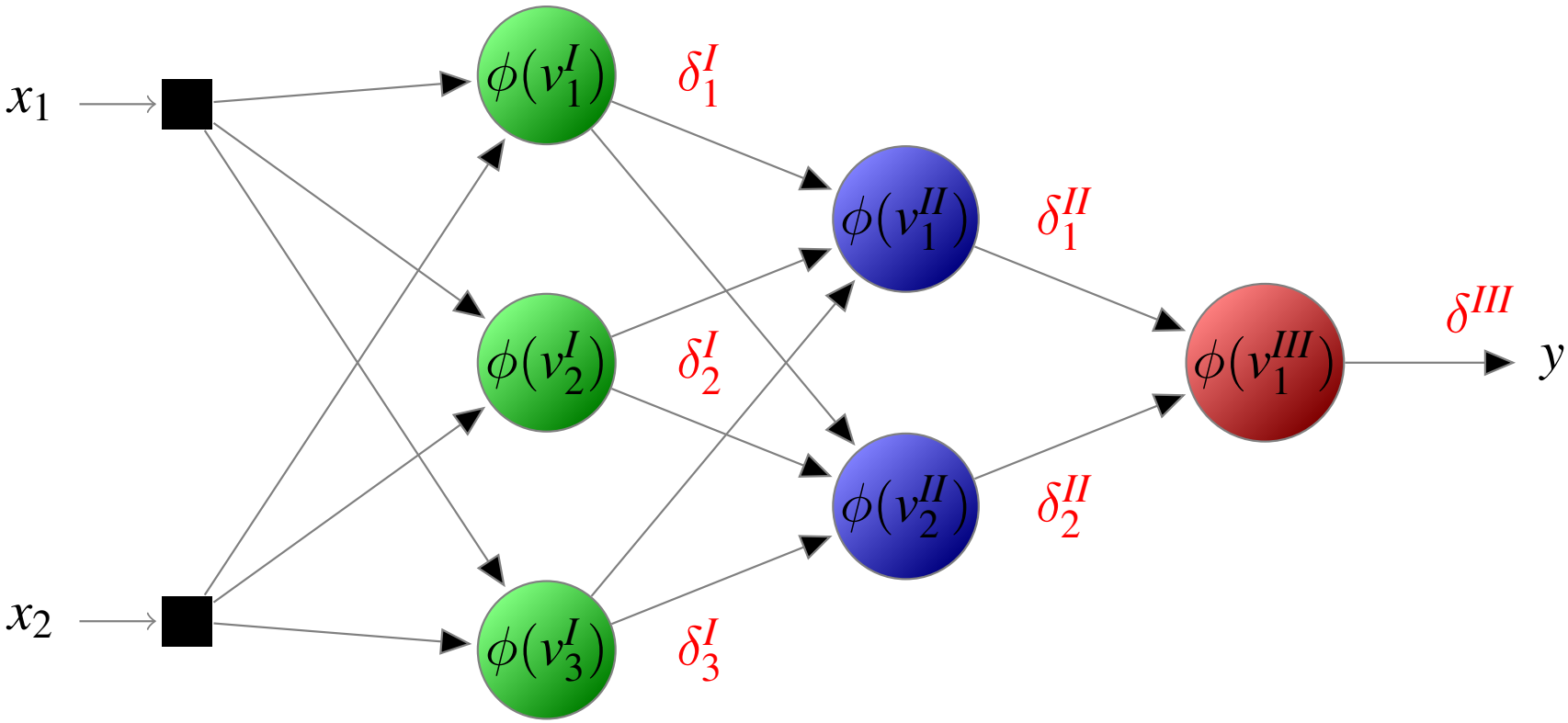
# Resumen: propagación hacia atras



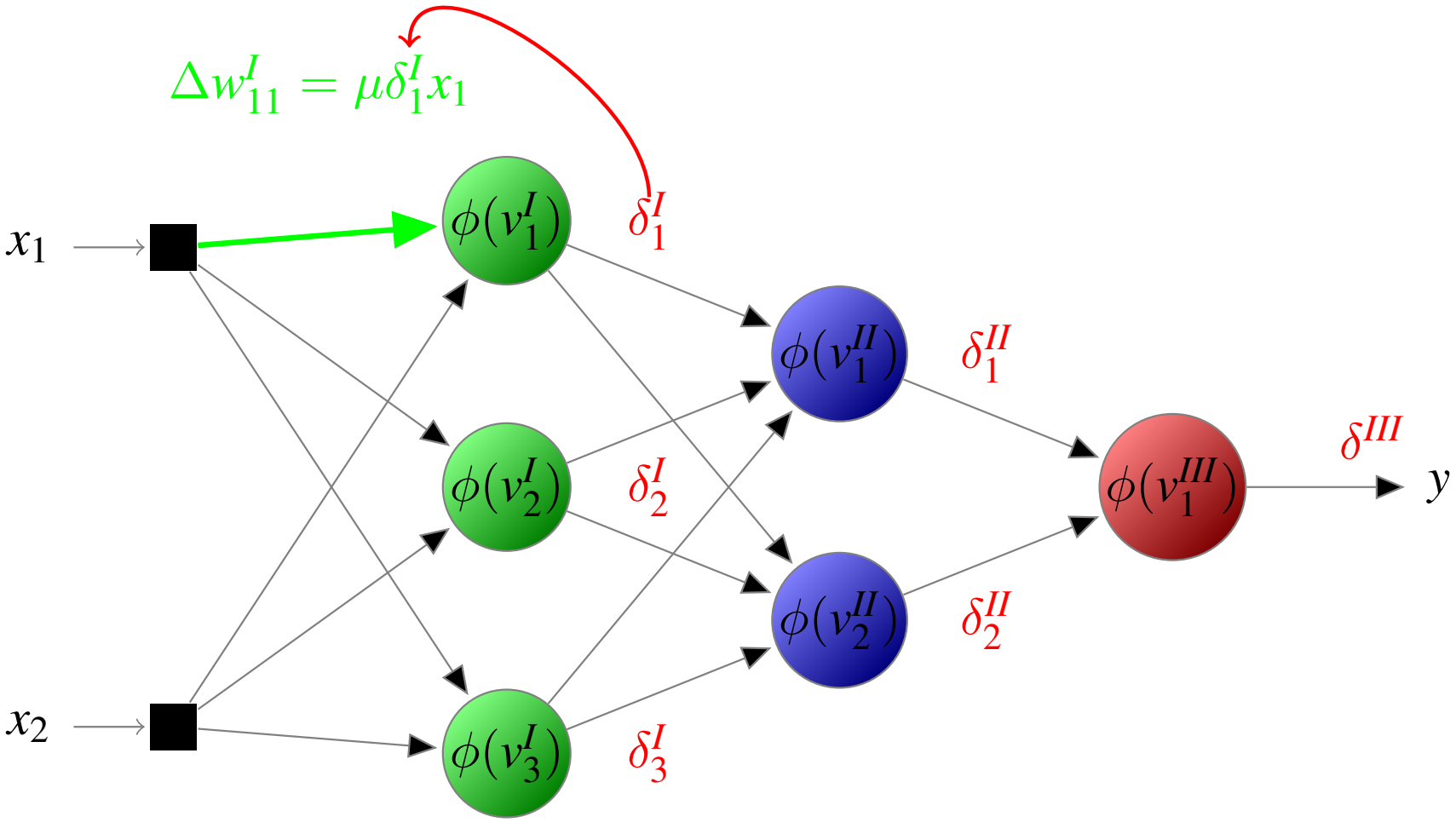
# Resumen: propagación hacia atras



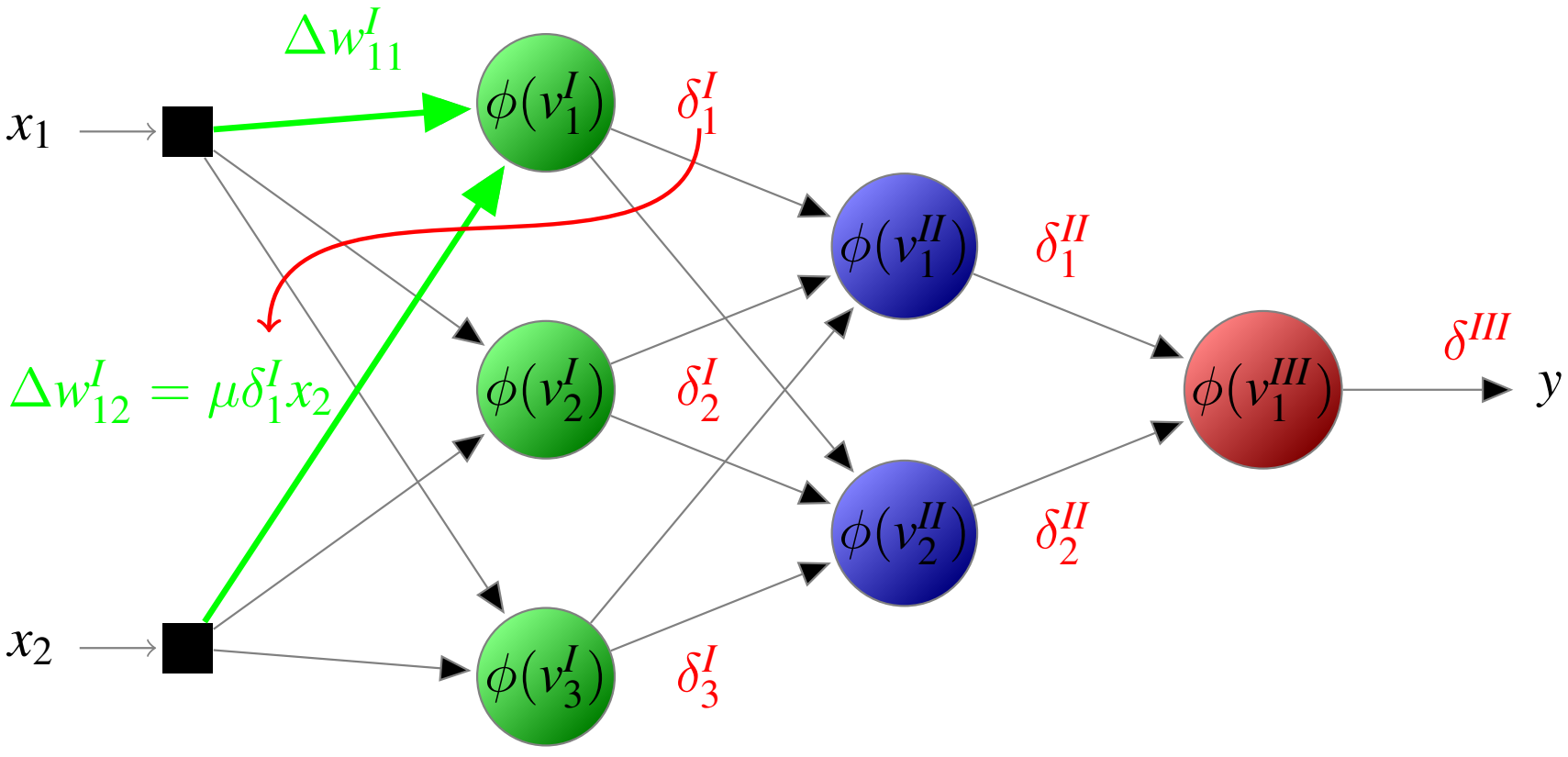
# Resumen: ajuste de pesos



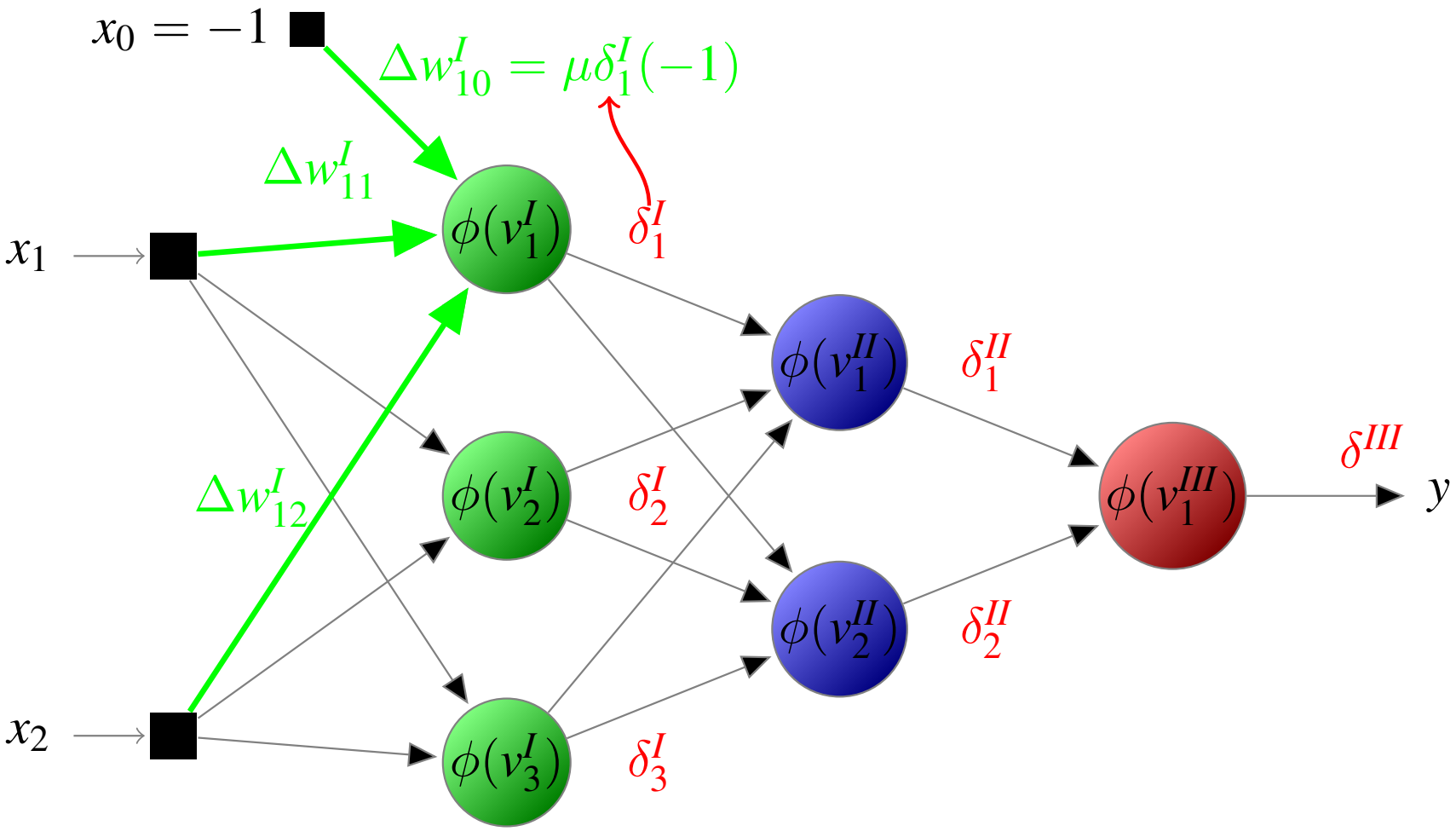
# Resumen: ajuste de pesos



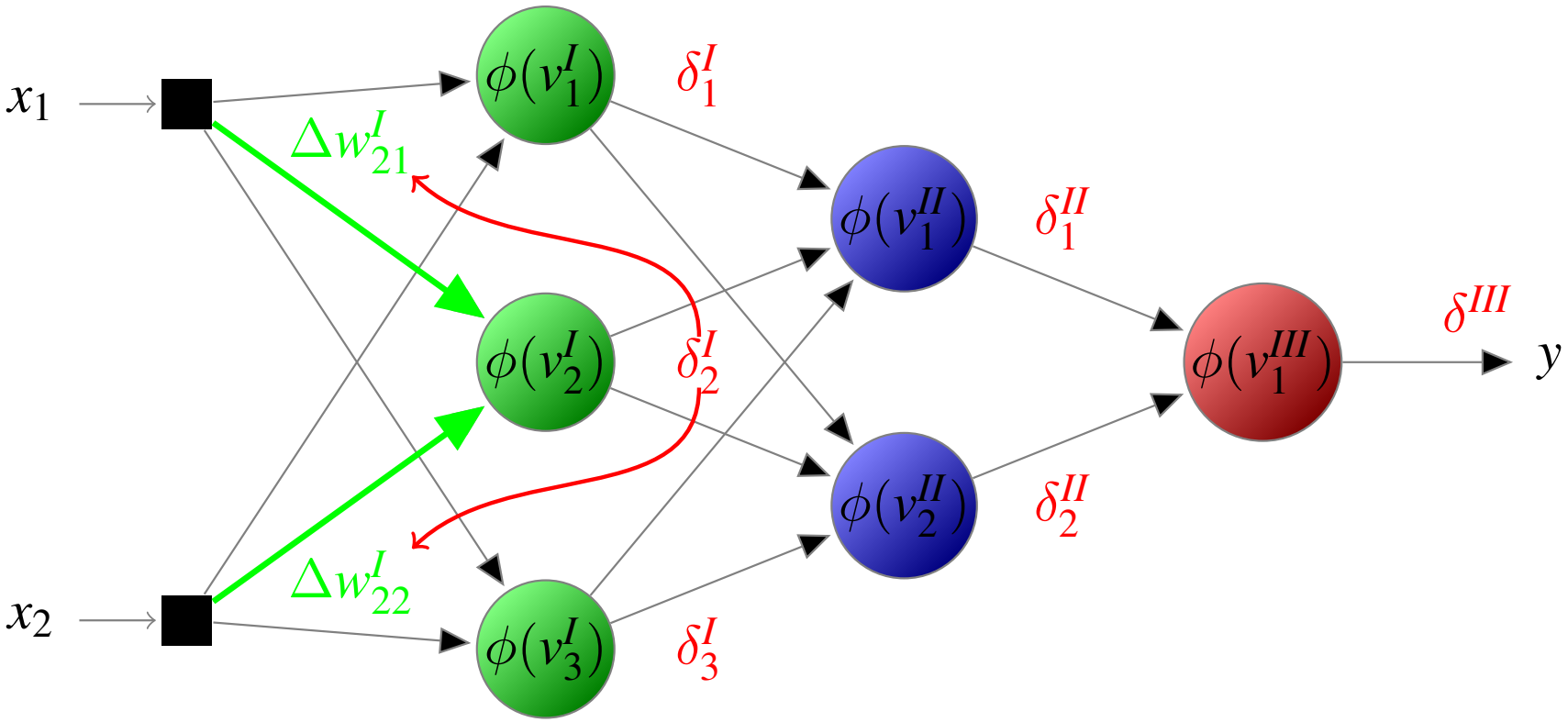
# Resumen: ajuste de pesos



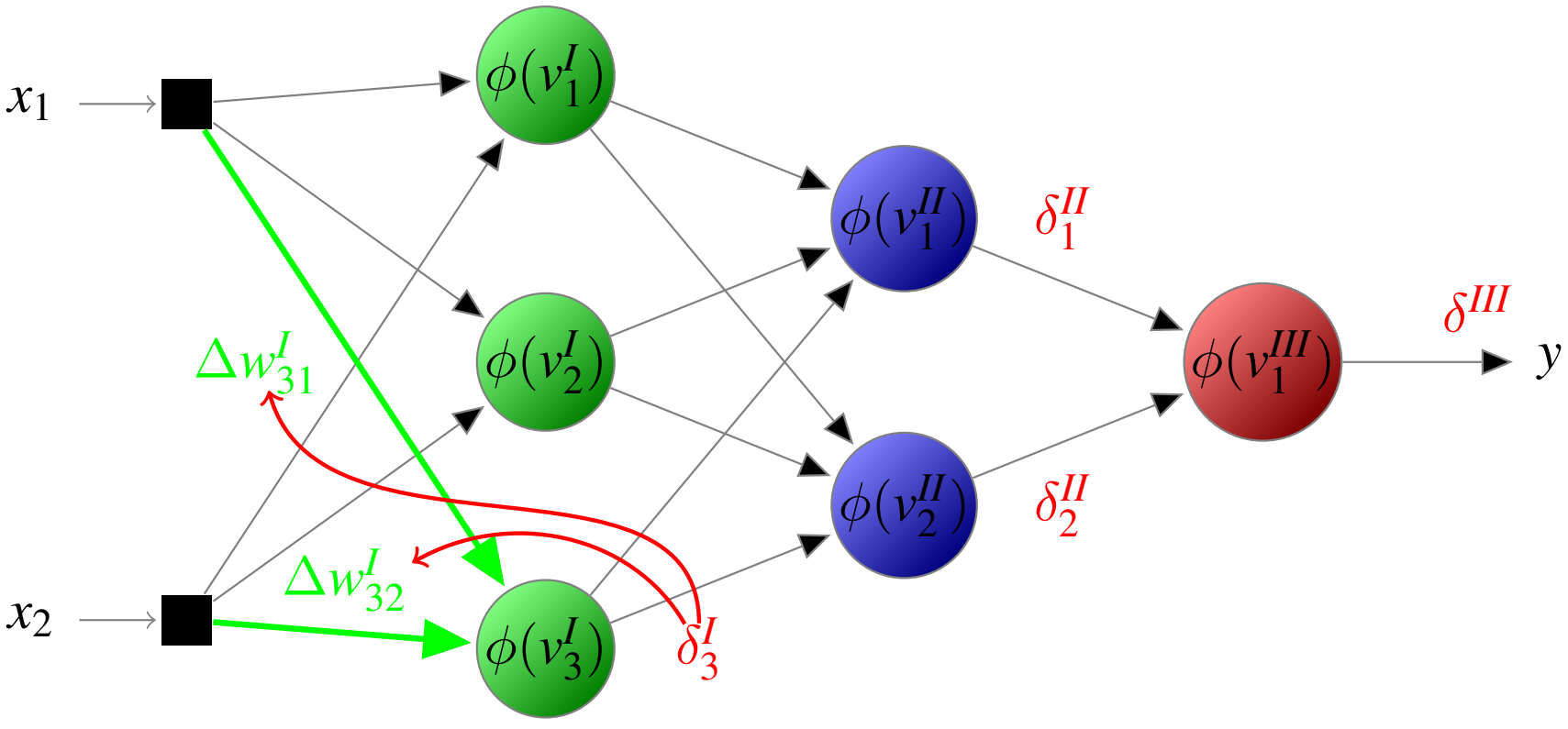
# Resumen: ajuste de pesos



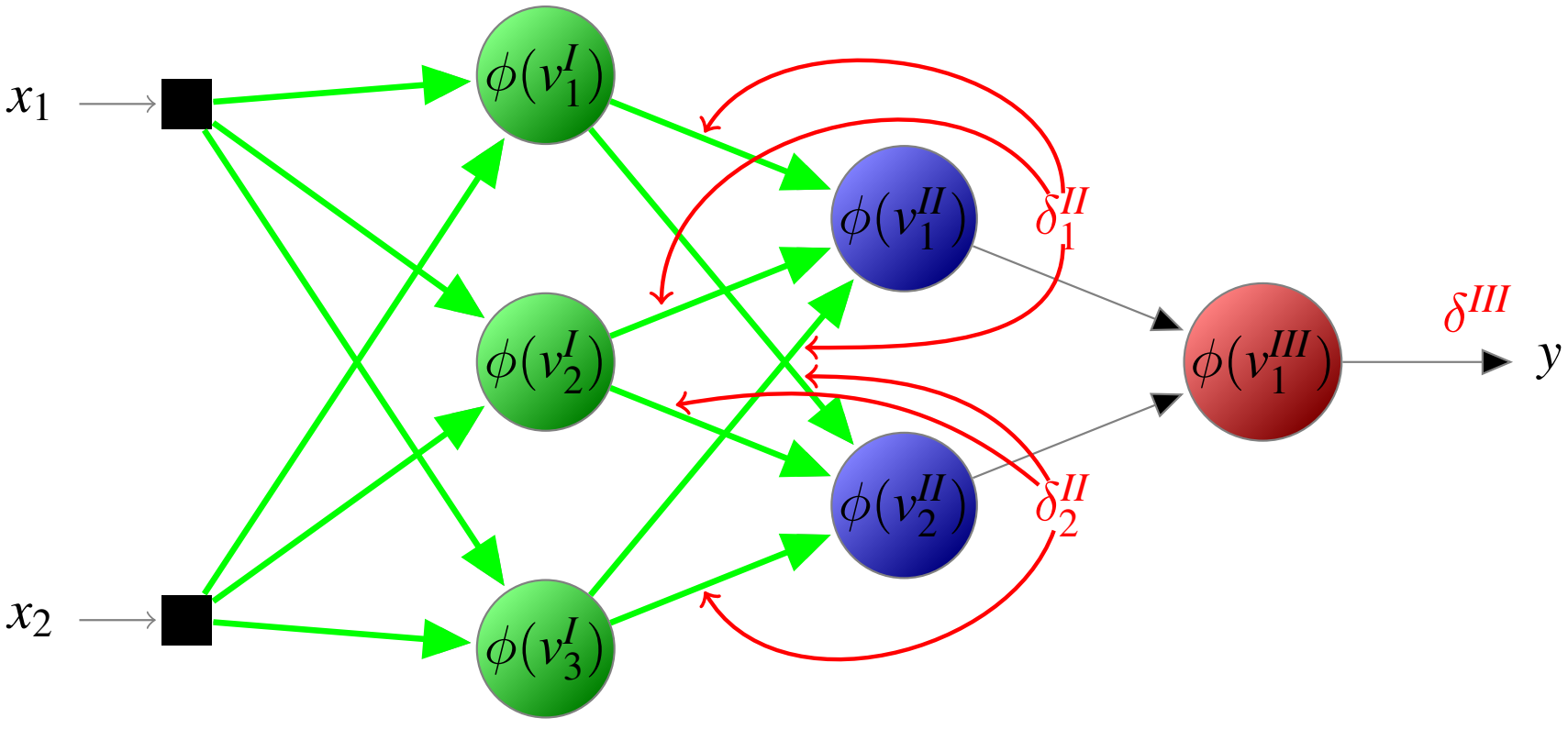
# Resumen: ajuste de pesos



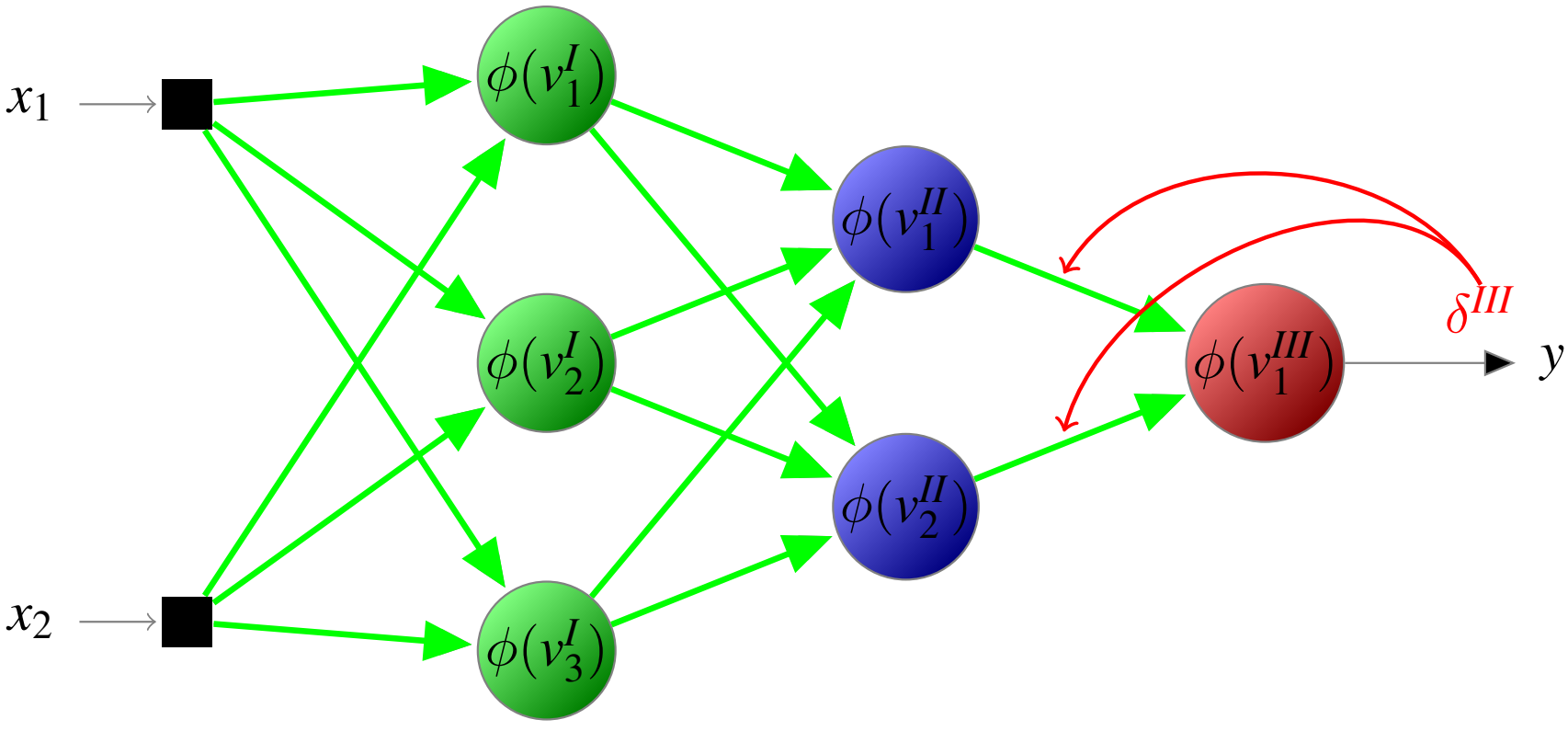
# Resumen: ajuste de pesos



# Resumen: ajuste de pesos



# Resumen: ajuste de pesos



# Resumen: ajuste de pesos

